Essentials of Machine Learning

ABU DHABI SCHOOL

ON COMPUTATIONAL MATERIALS DISCOVERY

Tanujit Chakraborty

SORBONNE UNIVERSITY ABU DHABI

scai SORBONNE CENTER FOR ARTIFICIAL INTELLIGENCE

DukeNUS Medical School

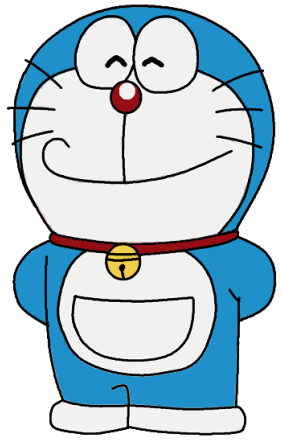
CIDSAI

NYU ABU DHABI

Team and Outline



Tanujit Chakraborty
Research Interests: Machine learning, Health Data Science, Network Science, etc.



Doraemon
Fictional character

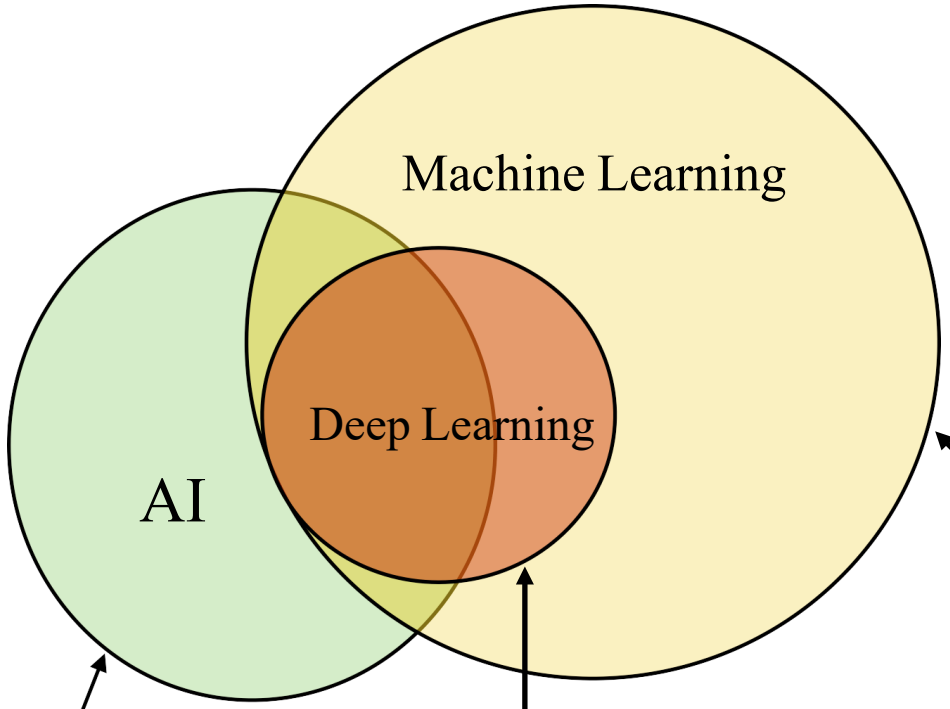


Shin-chan
Fictional character

Outline:

- Introduction to ML
- Taxonomy of ML
- Linear Models
- Neural Networks
- Graph Neural Networks
- Physics-Informed Networks
- References

Machine Learning, AI, and Deep Learning



A non-biological system that is intelligent through rules

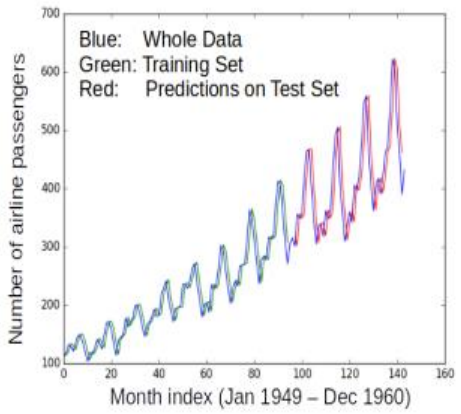
Algorithms that parameterize multilayer neural networks that then learn representations of data with multiple layers of abstraction

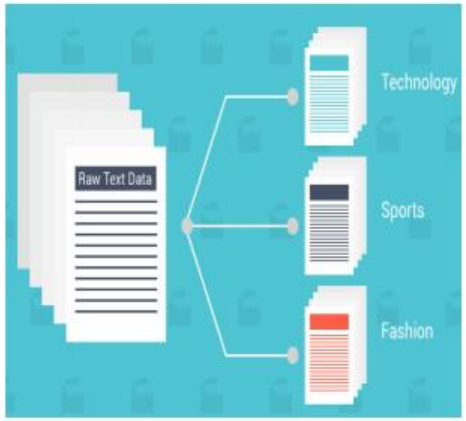
Algorithms that learn models/representations / rules automatically from data/examples

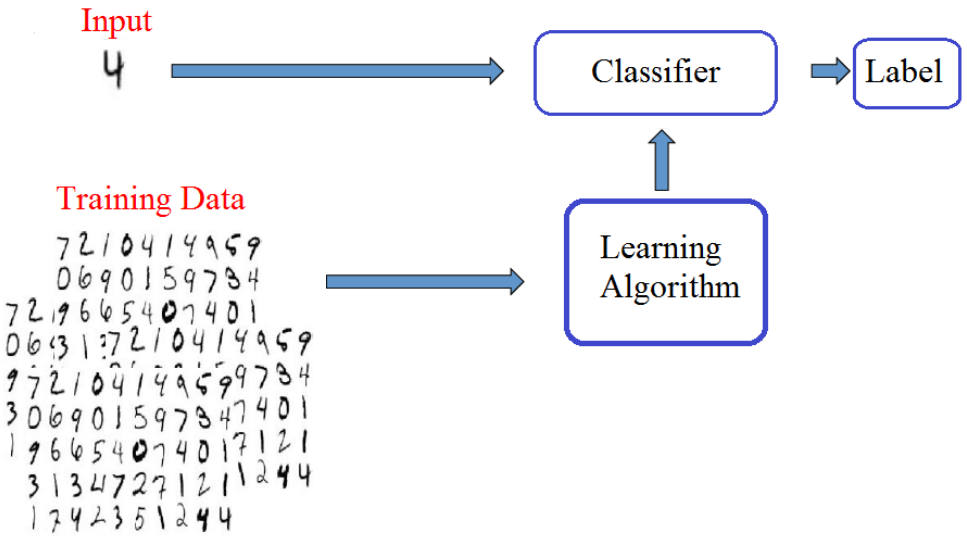
Machine Learning (ML)

- Designing algorithms that ingest data and learn a model of the data
- The learned model can be used to
  - Detect patterns/structures/themes/trends etc. in the data
  - Make predictions about future data and make decisions







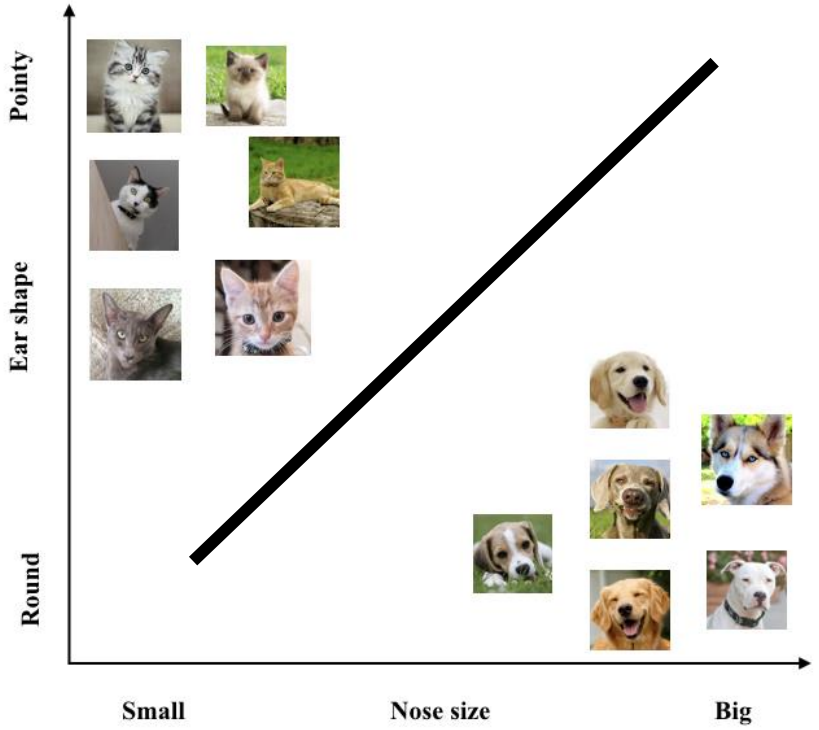


- Modern ML algorithms are heavily "data-driven"
  - No need to pre-define and hard-code all the rules (infeasible/impossible anyway)
  - The rules are not "static"; can adapt as the ML algorithm ingests more and more data

ML: From What It Does to How It Does It?

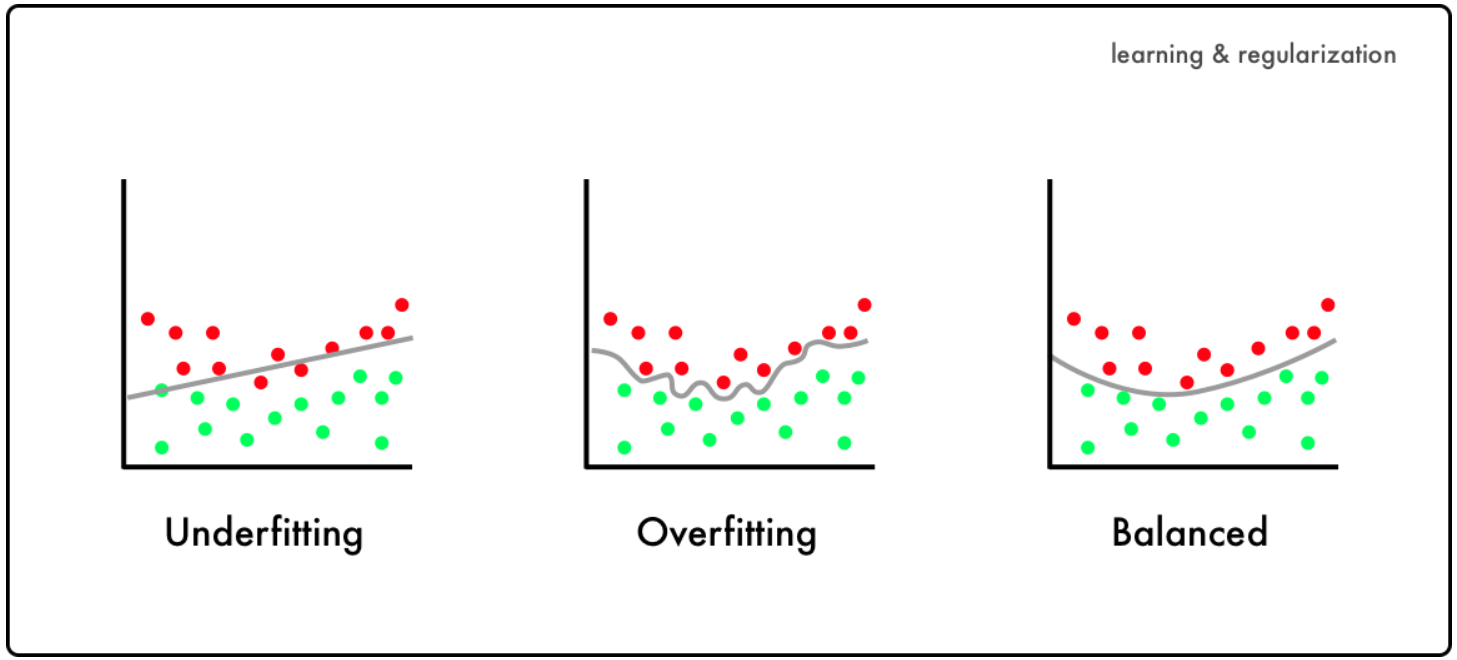
- ML enables intelligent systems to be data-driven rather than rule-driven
- How: By supplying training data and building statistical models of data
- Pictorial illustration of an ML model for binary classification:

A Linear Classifier (the statistical model)



Overfitting = Bad ML

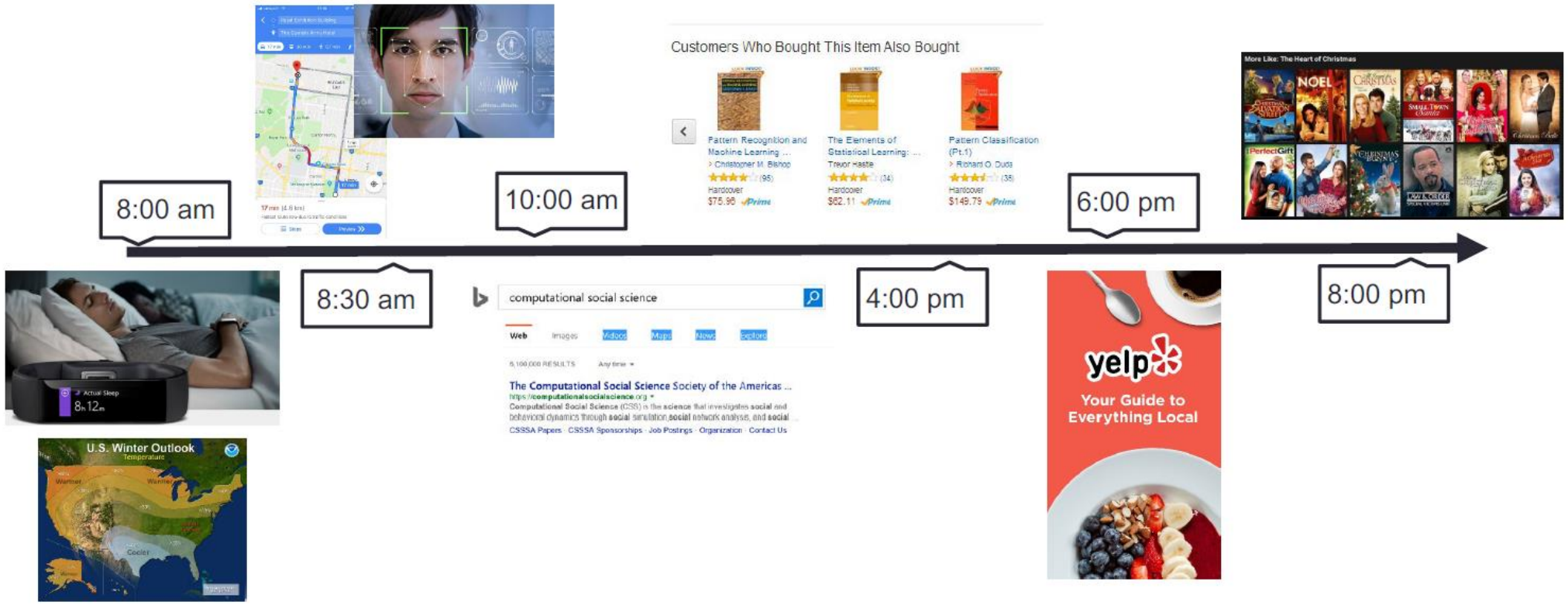
- Doing perfectly on training data is not good enough



- A good ML model must generalize well on unseen (test data)

- Simpler models should be preferred over more complex ones!

ML: Impacting our daily life



A Loose Taxonomy of ML

Learning using **labeled** data

Learning using **unlabeled** data

Some examples of supervised learning problems

- Classification
- Regression
- Ranking

Supervised Learning

Machine Learning

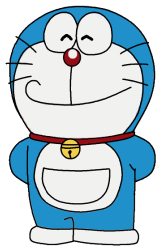
Unsupervised Learning

Some examples of unsupervised learning problems

- Clustering
- Dimensionality Reduction
- Unsupervised Probability Density Estimation

Reinforcement Learning

"Labeled" means, during training, for each input, the corresponding output is available (i.e., the machine learner is explicitly told that a cat image is of a cat)

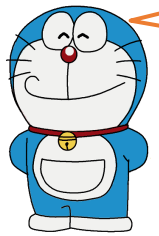


RL doesn't use "labeled" or "unlabeled" data in the traditional sense! In RL, an agent learns via its interactions with an environment

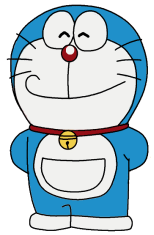
Many other specialized flavors of ML also exist, some of which include (not included in this talk):

- Semi-supervised Learning
- Active Learning
- Transfer Learning
- Multitask Learning
- Imitation Learning (somewhat related to RL)
- Zero-Shot Learning
- Few-Shot Learning
- Federated learning

A Typical Supervised Learning Workflow



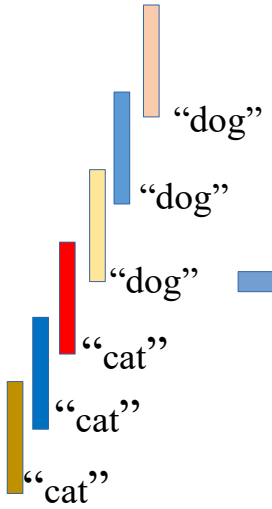
Note: This example is for the problem of binary classification, a supervised learning problem

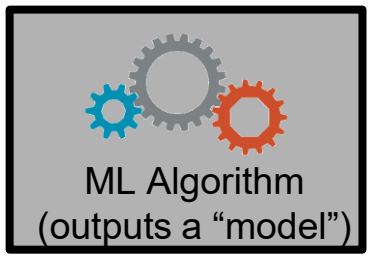


Labeled Training Data



"Feature" Extraction





Is feature extraction done "**manually**" as a pre-processing step before the ML algo starts working? Can't we "automate" this part? Can't we "learn" good features directly from raw inputs?



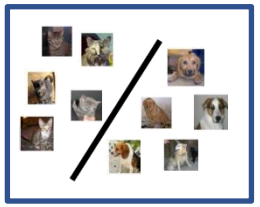
Feature extraction converts raw inputs to a numeric representation that the ML algo can understand and work with.

Indeed. Deep Learning algos do precisely that! (feature + model learning).



Test Image

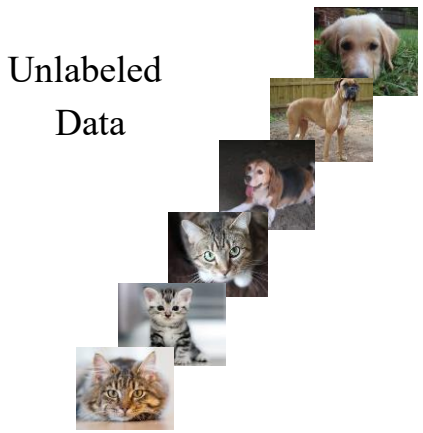
"Feature" Extraction

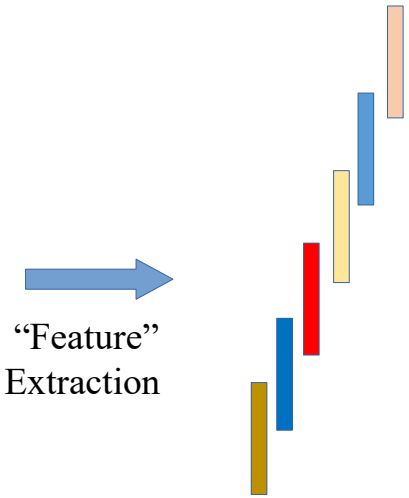


Cat vs Dog Prediction model

Predicted Label (cat/dog)

A Typical Unsupervised Learning Workflow

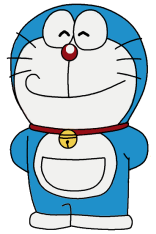




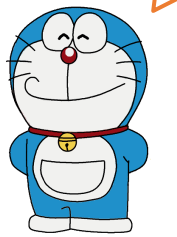




Note: This example is for the problem of data clustering, an unsupervised learning problem



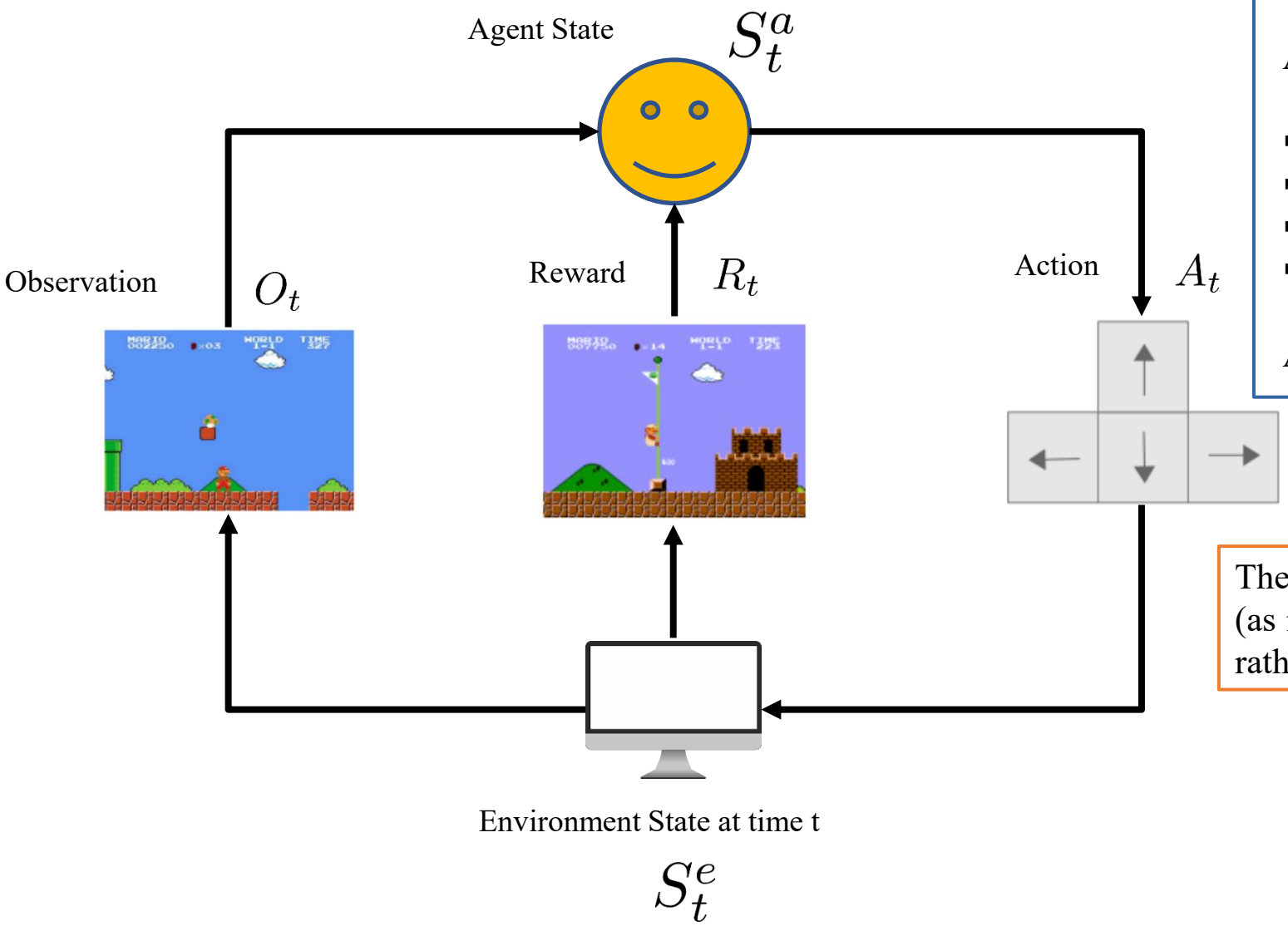
Yes. In this example, given a new "test" cat/dog image, we can assign it to the cluster with closer centroid





Does unsupervised learning also have a test phase? That is, can we also predict the cluster of a new test input?

A Typical Reinforcement Learning Workflow



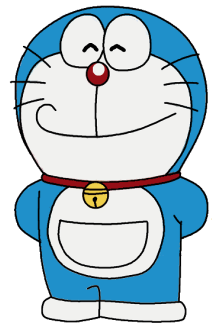
Wish to teach an agent optimal policy for some task

Agent does the following repeatedly

- Senses/observes the environment
- Takes an action based on its current policy
- Receives a reward for that action
- Updates its policy

Agent's goal is to maximize its overall reward

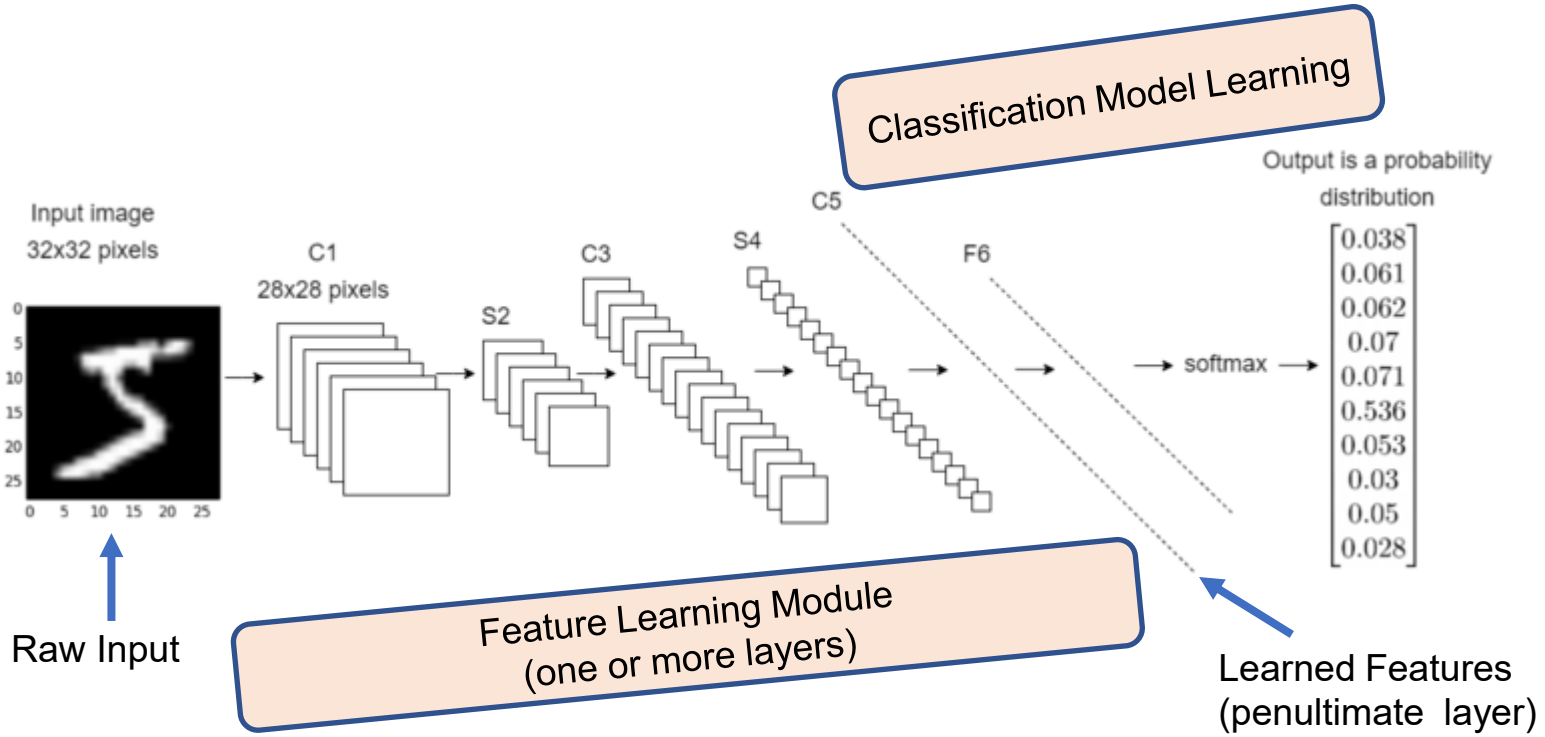
There is supervision, not explicit (as in Supervised Learning) but rather implicit (feedback based)



Deep Learning: An End-to-End Approach to ML

Deep Learning = ML with automated feature learning from the raw inputs

Feature extraction part is automated via the feature learning module

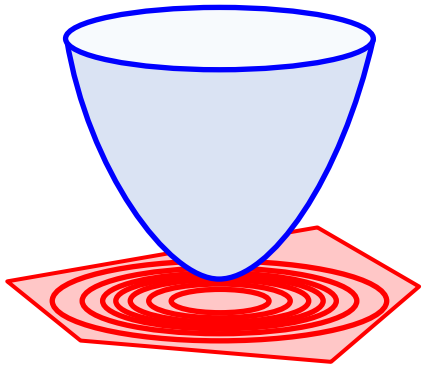


Pic an adaptation of the original from: https://deepai.org/

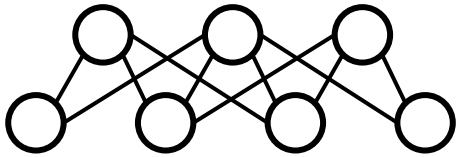
ML can store info in lots of innovative ways

ML Models and Algorithms

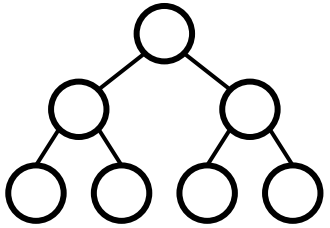
Linear/Optimization



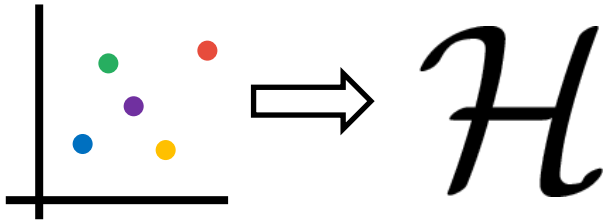
Neural/Deep

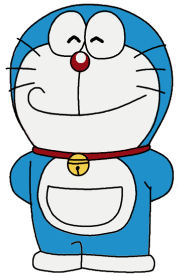


Local

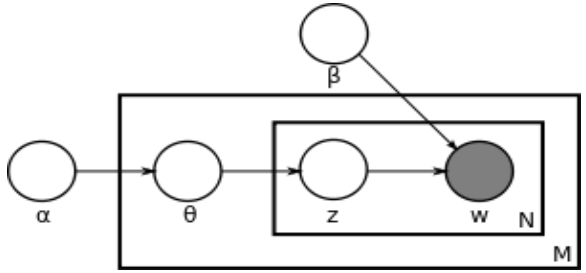


Kernel





Probabilistic/Bayesian



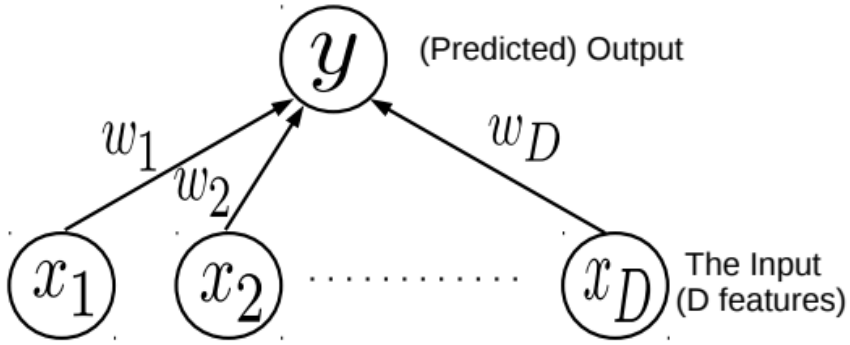
Linear Models



- Consider learning to map an input $\boldsymbol{x} \in \mathbb{R}^D$ to the corresponding (say real-valued, e.g., properties of material) output y (e.g., a continuous physical property like Young's modulus)

- Assume the output to be a linear weighted combination of the D input features

$$y = \sum_{d=1}^{D} w_d x_d = \boldsymbol{w}^\top \boldsymbol{x}$$



This defines a linear model with D parameters given by a "**weight vector**" $\boldsymbol{w} = [w_1, w_2, \ldots, w_D]$

Each of these weights have a simple interpretation: w_d is the "**weight**" or importance of the dth feature in making this prediction

The "optimal" weights are unknown and have to be learned by solving an **optimization problem**, using some training data **(minimizing loss)**

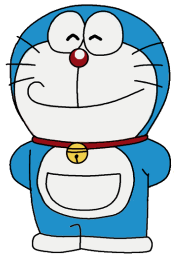
- This simple model can be used for Linear Regression

- This simple model can also be used as a "**building block**" for more complex models
  - Even classification (binary/multi-class/multi-output/multi-label) and various other ML/DL models.
  - Even unsupervised learning problems (e.g., dimensionality reduction models)

Loss Functions for Regression

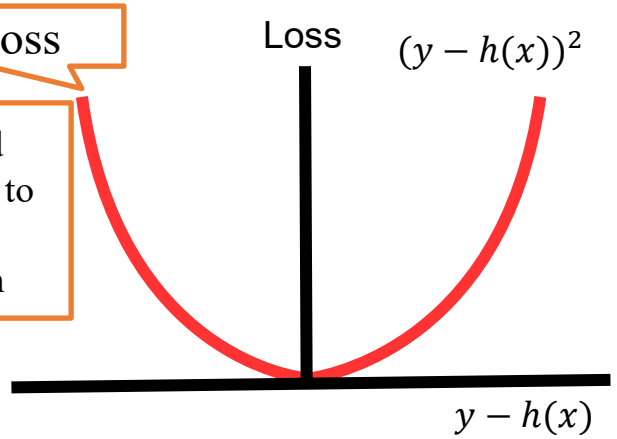
- Many possible loss functions for regression problems

Choice of loss function usually depends on the nature of the data. Also, some loss functions result in easier optimization problem than others



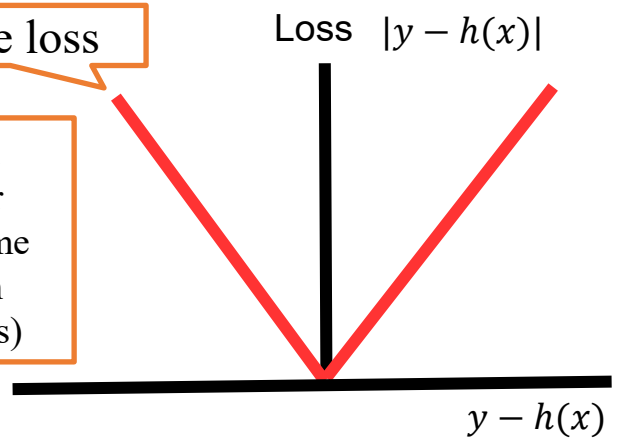
Squared loss

Very commonly used for regression. Leads to an easy-to-solve optimization problem



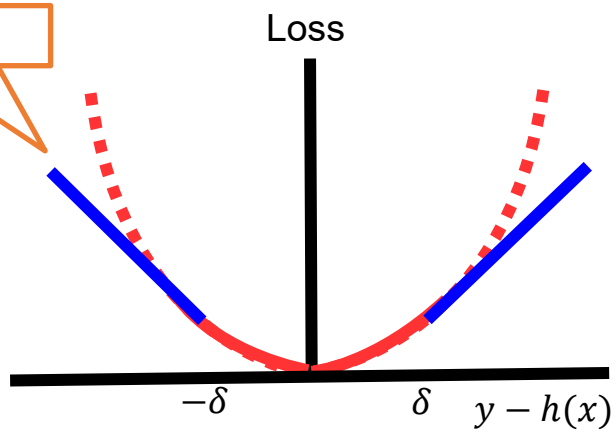
Absolute loss

Grows more slowly than squared loss. Thus better suited when data has some outliers (inputs on which model makes large errors)



Huber loss

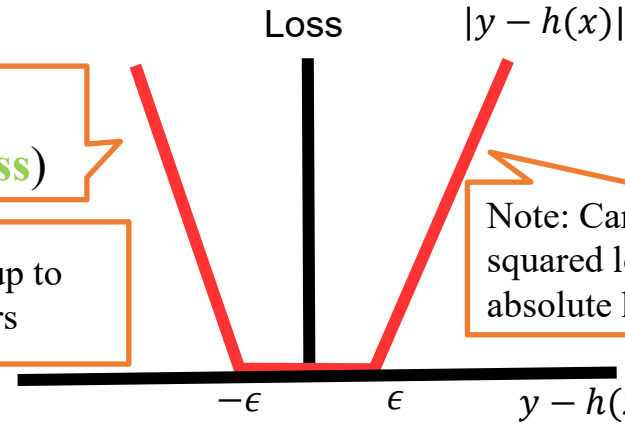
Squared loss for small errors (say up to δ); absolute loss for larger errors. Good for data with outliers



$$L_\delta(y, h(x)) = \begin{cases} \frac{1}{2}(y - h(x))^2 & \text{for } |y - h(x)| \le \delta, \\ \delta|y - h(x)| - \frac{1}{2}\delta^2 & \text{otherwise.} \end{cases}$$

ϵ-insensitive loss (a.k.a. Vapnik loss)

Zero loss for small errors (say up to ϵ); absolute loss for larger errors

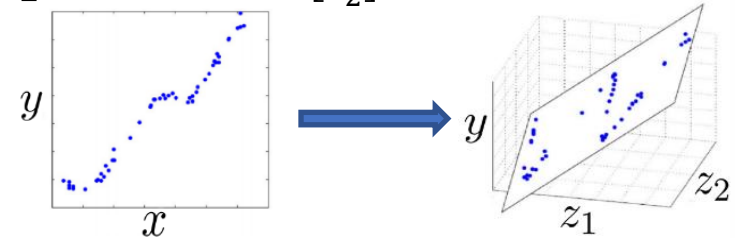


Note: Can also use squared loss instead of absolute loss

Linear Regression: Pictorially

- Linear regression is like fitting a line or (hyper)plane to a set of points

$$\begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \phi(x)$$



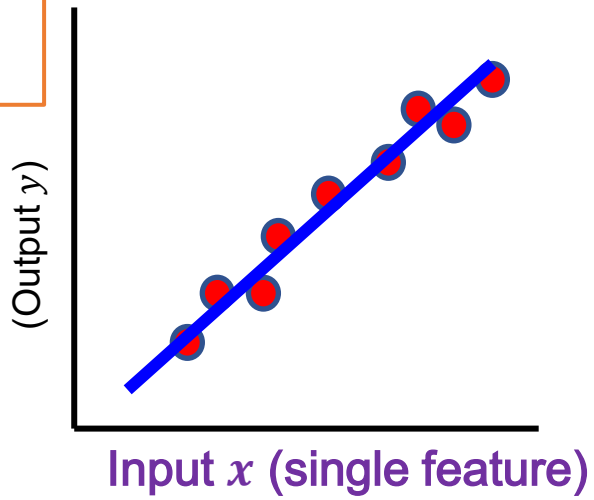
Original (single) feature
Nonlinear curve needed

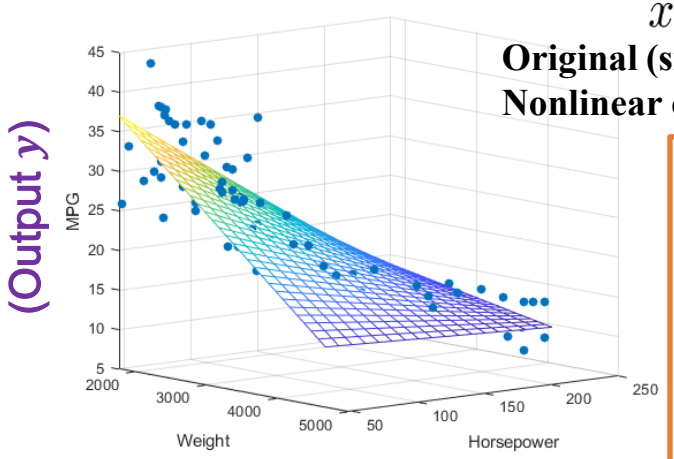
Two features
Can fit a plane (linear)

What if a line/plane doesn't model the input-output relationship very well, e.g., if their relationship is better modeled by a nonlinear curve or curved surface?



Do linear models become useless in such cases?





No. We can even fit a curve using a linear model after suitably transforming the inputs $y \approx \boldsymbol{w}^\top \boldsymbol{\phi}(\boldsymbol{x})$

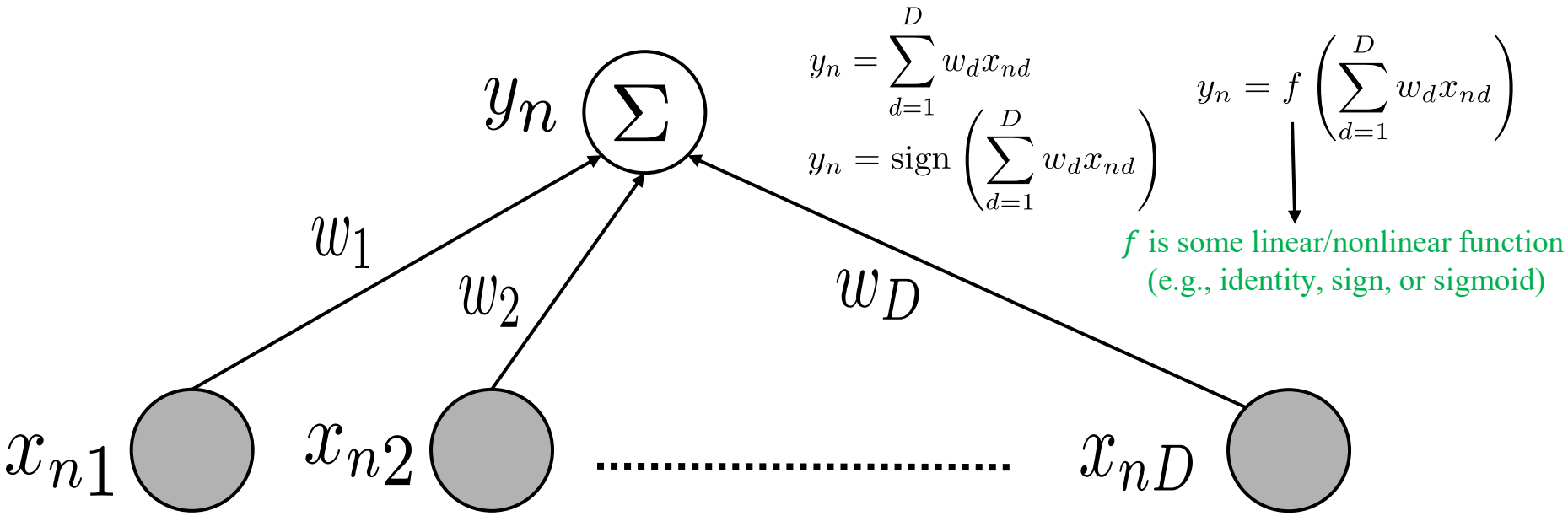


The transformation $\phi(.)$ can be predefined or learned (e.g., using kernel methods or a deep neural network based feature extractor)

- The line/plane must also predict outputs the unseen (test) inputs well !

Limitation of Linear Models

- Linear models: Output produced by taking a linear combination of input features

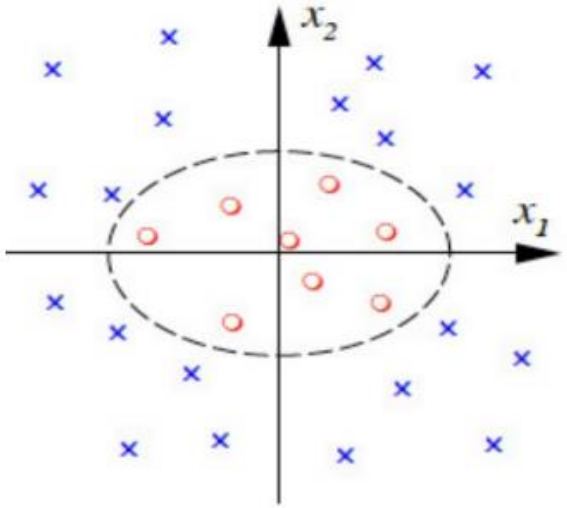


$$y_n = \sum_{d=1}^{D} w_d x_{nd}$$

$$y_n = \text{sign}\left(\sum_{d=1}^{D} w_d x_{nd}\right)$$

$$y_n = f\left(\sum_{d=1}^{D} w_d x_{nd}\right)$$

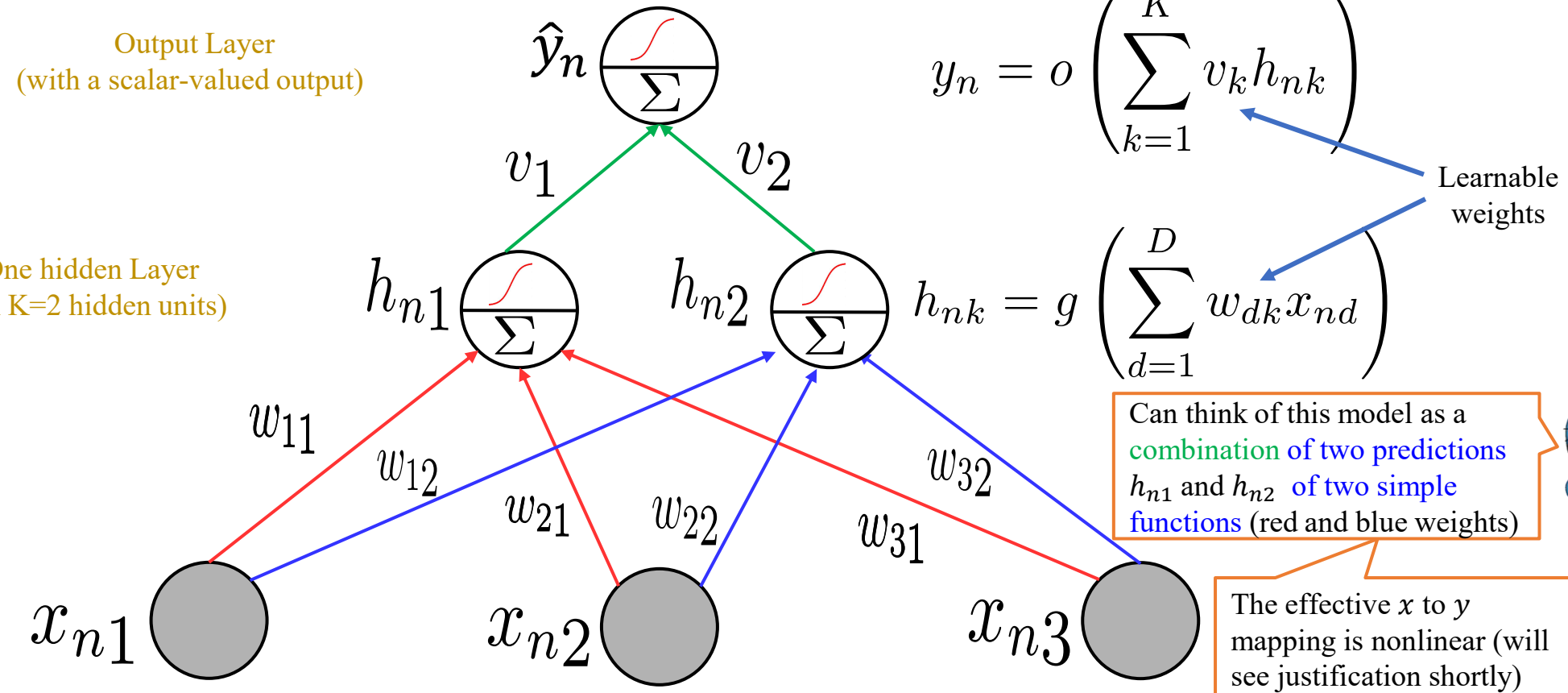
f is some linear/nonlinear function
(e.g., identity, sign, or sigmoid)



- A basic unit of the form $y = f(\boldsymbol{w}^\top \boldsymbol{x})$ is known as the "Perceptron"
- Goal: Find $\boldsymbol{w}$
- Backpropagation = Gradient descent using chain rule of derivatives

Neural Networks: Multi-layer Perceptron (MLP)

- An MLP is a network containing several Perceptron units across many layers
- An MLP consists of an input layer, an output layer, and one or more hidden layers



MLP is also called feedforward fully-connected network

Illustration: Neural Net with Single Hidden Layer

- Compute K pre-activations for each input $\boldsymbol{x}_n$

A linear model with learnable weight vector $\boldsymbol{w}_k$

$$z_{nk} = \boldsymbol{w}_k^\top \boldsymbol{x}_n = \sum_{d=1}^{D} w_{dk} x_{nd} \quad (k = 1,2,\dots,K)$$

- Apply nonlinear activation on each pre-act

Called a hidden unit

$$h_{nk} = g(z_{nk}) \quad (k = 1,2,\dots,K)$$

- Apply a linear model with $\boldsymbol{h}_n$ acting as features

A linear model with learnable weight vector $\boldsymbol{v}$

Score of the input

$$s_n = \boldsymbol{v}^\top \boldsymbol{h}_n = \sum_{k=1}^{K} v_k h_{nk}$$

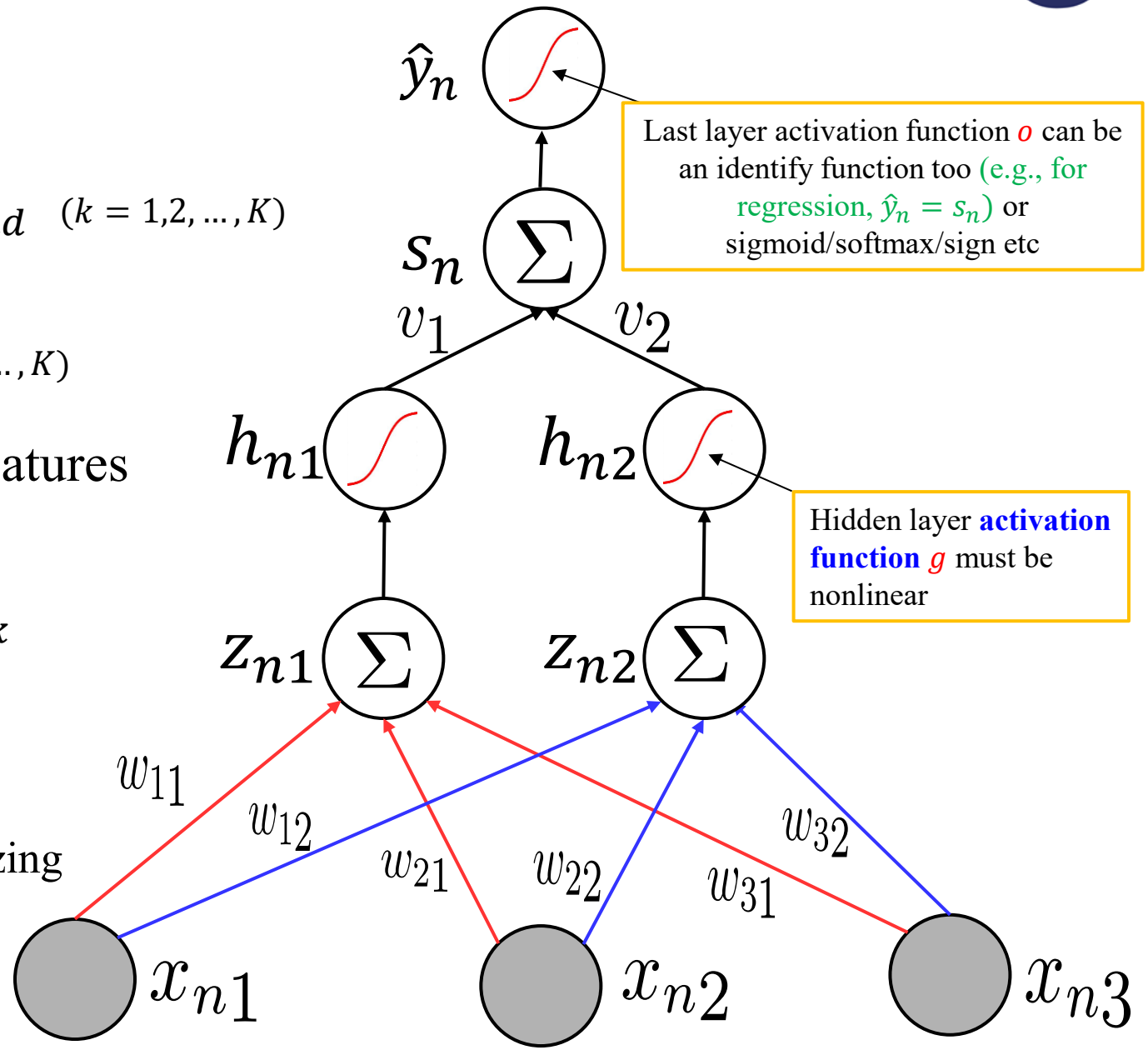
- Finally, output is produced as

Score converted to the actual prediction

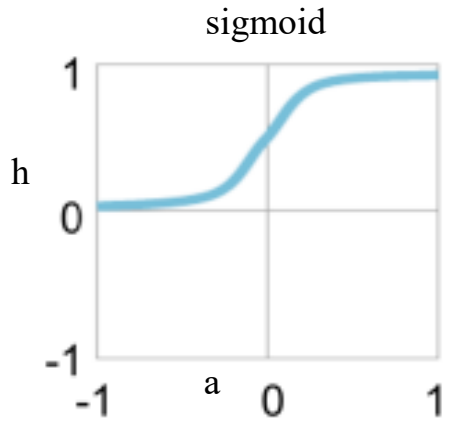
$$\hat{y}_n = o(s_n)$$

- Unknowns $(\boldsymbol{w}_1, \boldsymbol{w}_2, \dots, \boldsymbol{w}_K, \boldsymbol{v})$ learned by minimizing some loss function (e.g., squared)

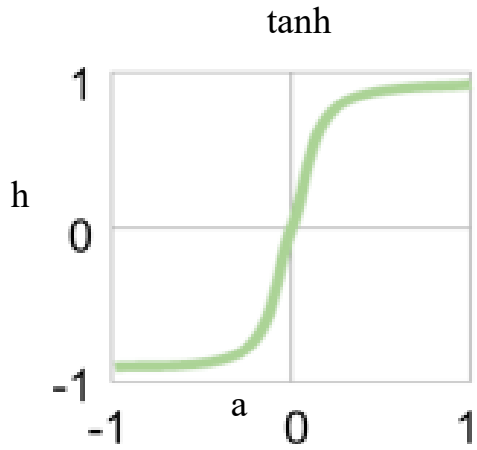
Loss: $\mathcal{L}(\boldsymbol{W}, \boldsymbol{v}) = \sum_{n=1}^{N} \ell(y_n, \hat{y}_n)$



Activation Functions: Some Common Choices



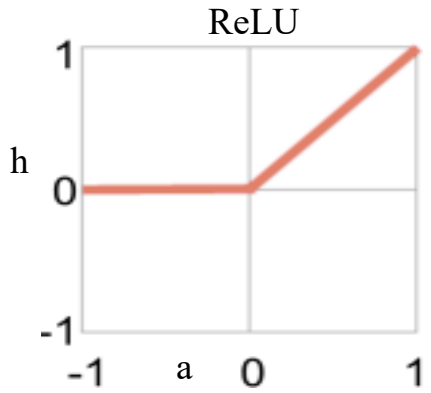
For sigmoid as well as tanh, gradients saturate (become close to zero as the function tends to its extreme values)



Helps keep the mean of the next layer's inputs close to zero (with sigmoid, it is close to 0.5)

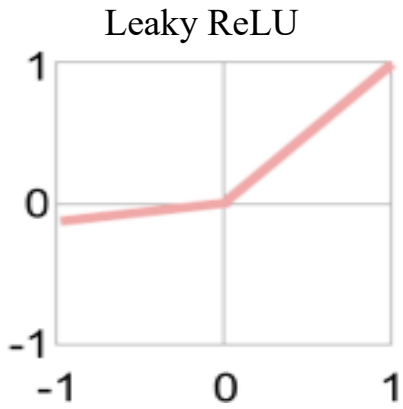
Sigmoid: $h = \sigma(a) = \frac{1}{1+\exp(-a)}$

tanh (tan hyperbolic): $h = \frac{\exp(a)-\exp(-a)}{\exp(a)+\exp(-a)} = 2\sigma(2a) - 1$



ReLU and Leaky ReLU are among the most popular ones (also efficient to compute)

Helps fix the dead neuron problem of ReLU when a is a negative number



$y = \boldsymbol{v}^\top(g(\boldsymbol{W}^\top x))$
$= \boldsymbol{v}^\top \boldsymbol{W}^\top x$

Still linear

Without nonlinear activation, a deep neural network is equivalent to a linear model no matter how many layers we use

Most activation functions are monotonic but there exist some non-monotonic activation functions as well (e.g., Swish: $a \times \sigma(\beta a)$)

ReLU (Rectified Linear Unit): $h = \max(0, a)$

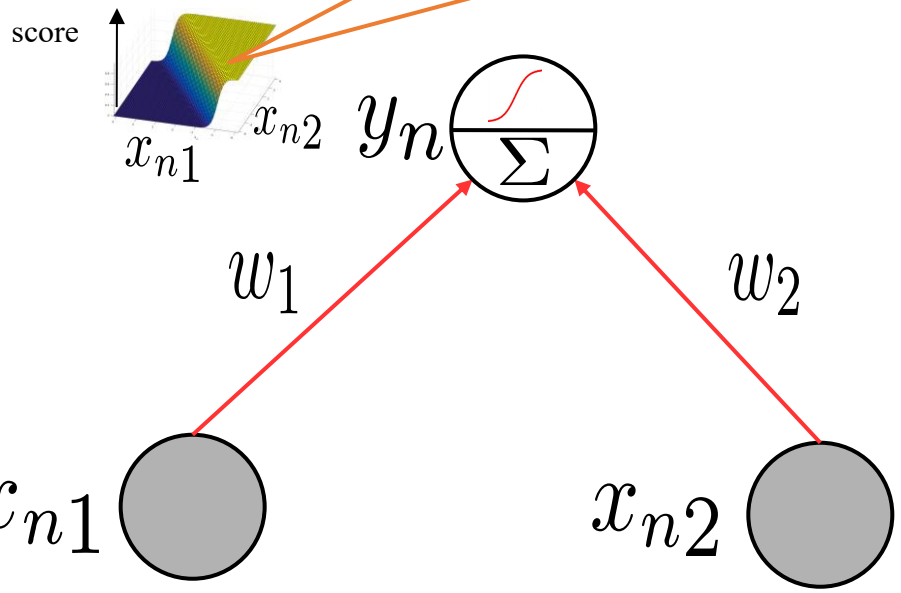
Leaky ReLU: $h = \max(\beta a, a)$
where β is a small postive number



MLP Can Learn Any Nonlinear Function

- An MLP can be seen as a composition of multiple linear models combined nonlinearly

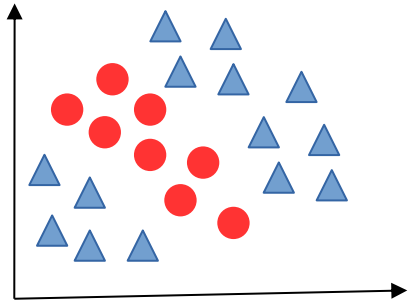
Score monotonically increases. One-sided increase (not ideal for learning nonlinear decision boundaries). Just a linear model



Standard Single "Perceptron" Classifier (no hidden units)

A single hidden layer MLP with sufficiently large number of hidden units can approximate any function (Hornik, 1991)

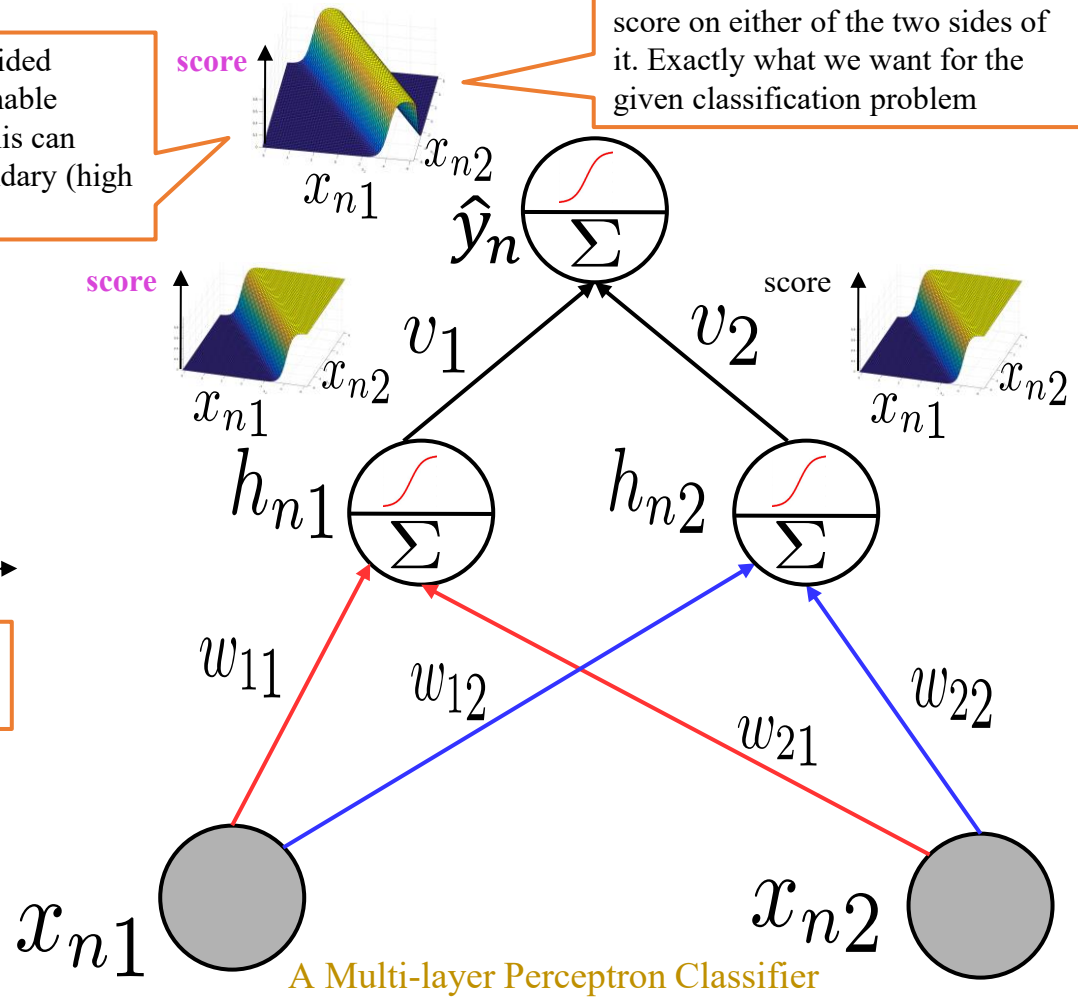




A nonlinear classification problem

Obtained by composing the two one-sided increasing score functions (using learnable weights v_1 and v_2 to "add" them). This can now learn the nonlinear decision boundary (high score in the middle)

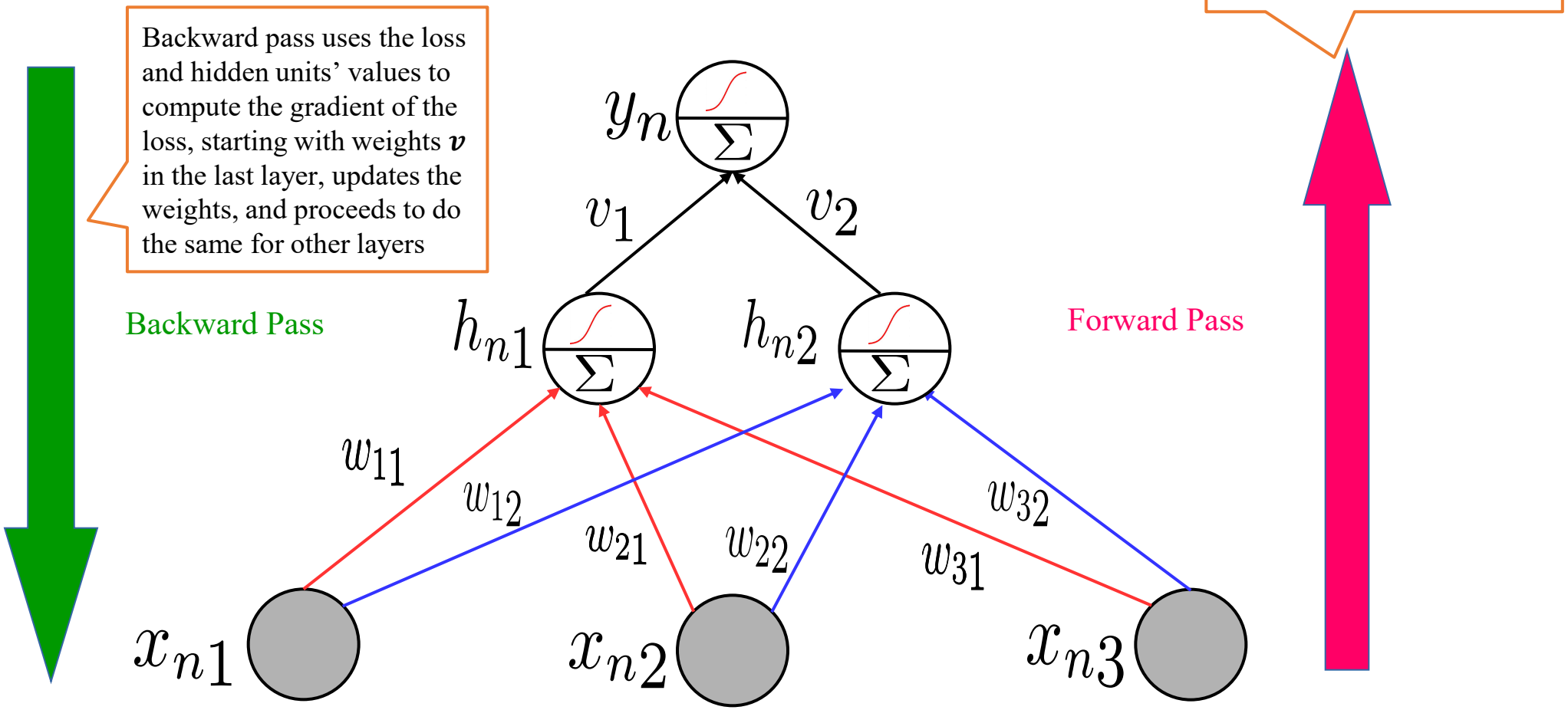
High-score in the middle and low-score on either of the two sides of it. Exactly what we want for the given classification problem



A Multi-layer Perceptron Classifier
(one hidden layer with 2 units)
Capable of learning nonlinear boundaries

Backpropagation

- Backprop iterates between a forward pass and a backward pass

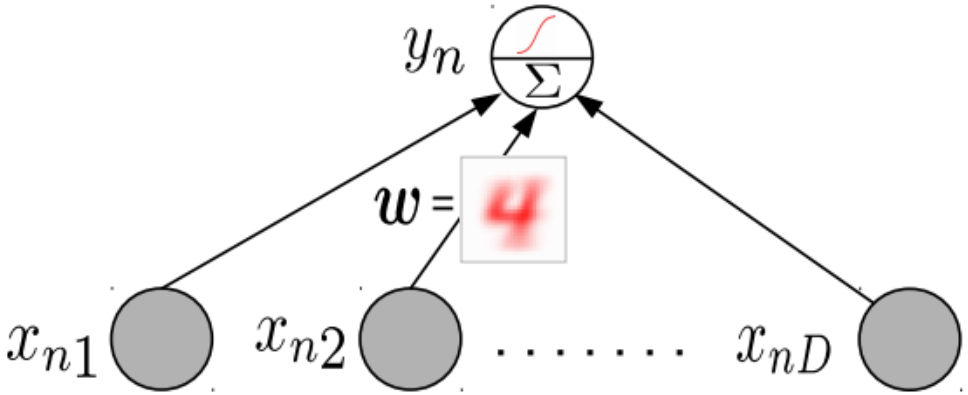


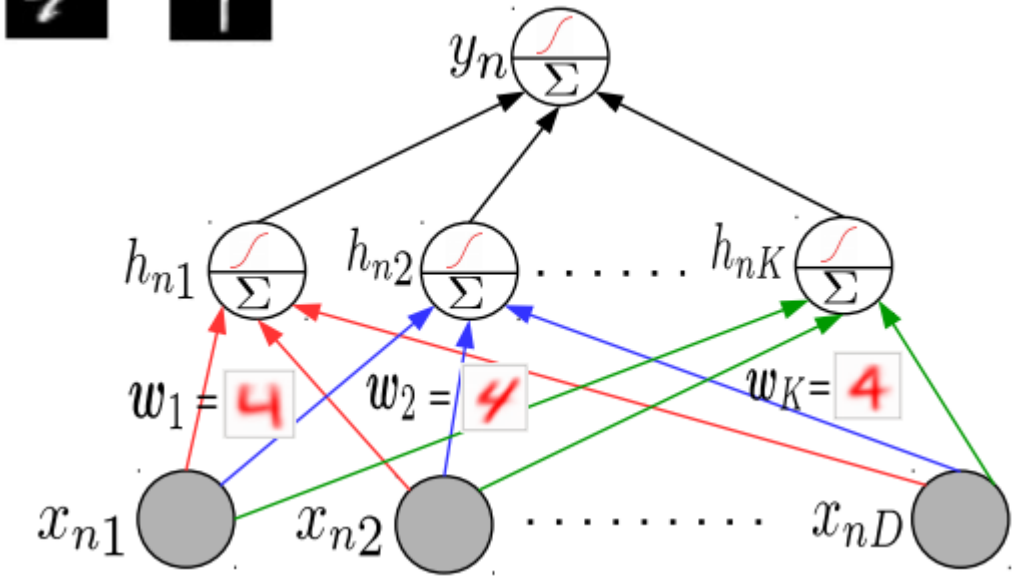
- Software frameworks such as Tensorflow and PyTorch support this

Why Neural Networks Work Better: Another View

- Linear models tend to only learn the "average" pattern
  - E.g., Weight vector of a linear classification model represent average pattern of a class
- Deep models can learn multiple patterns (each hidden node can learn one pattern)
  - Thus deep models can learn to capture more subtle variations that a simpler linear model







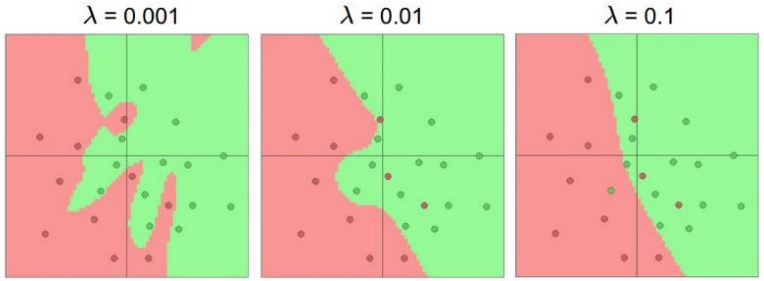
Preventing Overfitting in Neural Nets

Various other tricks, such as weight sharing across different hidden units of the same layer (used in convolutional neural nets or CNN)

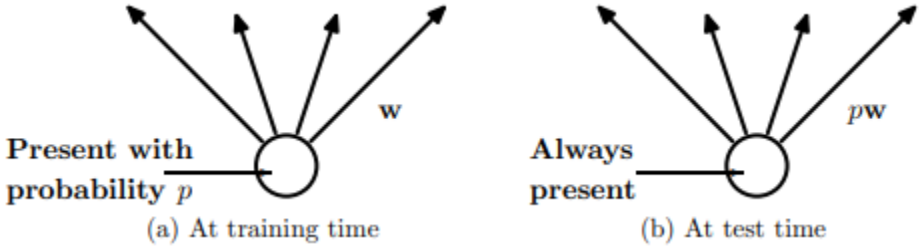


- Neural nets can overfit. Many ways to avoid overfitting, such as
  - Standard regularization on the weights, such as ℓ_2, ℓ_1, etc (ℓ_2 reg. is also called weight decay)

Single Hidden Layer NN with K = 20 hidden units and L2 regularization



  - Early stopping (traditionally used): Stop when validation error starts increasing
  - Dropout: Randomly remove units (with some probability $p \in (0,1)$) during training



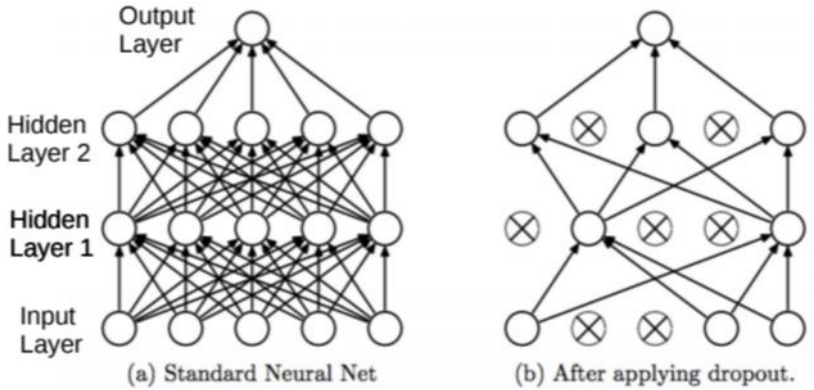


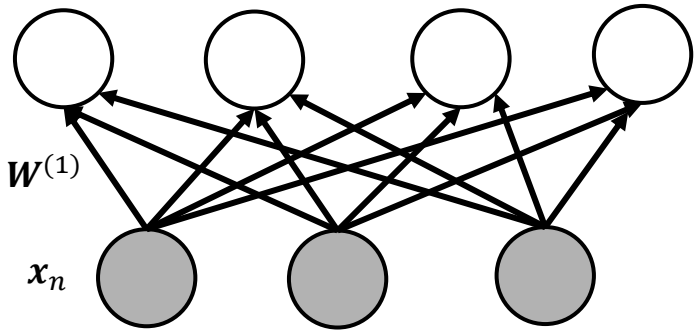
Fig courtesy: Dropout: A Simple Way to Prevent Neural Networks from Overfitting (Srivastava et al, JMLR 2014)

Limitations/Shortcomings of MLP

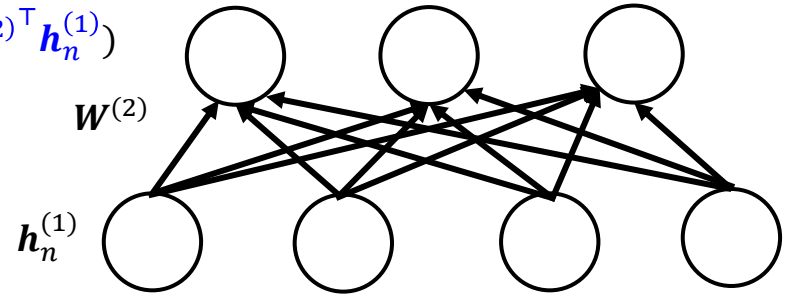
Projection using a "linear layer" + element-wise nonlinearity applied on these linear projections

- MLP uses fully connected layers defined by matrix multiplications + nonlinearity

$\boldsymbol{h}_n^{(1)} = g(\boldsymbol{W}^{(1)^\top}\boldsymbol{x}_n)$



$\boldsymbol{h}_n^{(2)} = g(\boldsymbol{W}^{(2)^\top}\boldsymbol{h}_n^{(1)})$

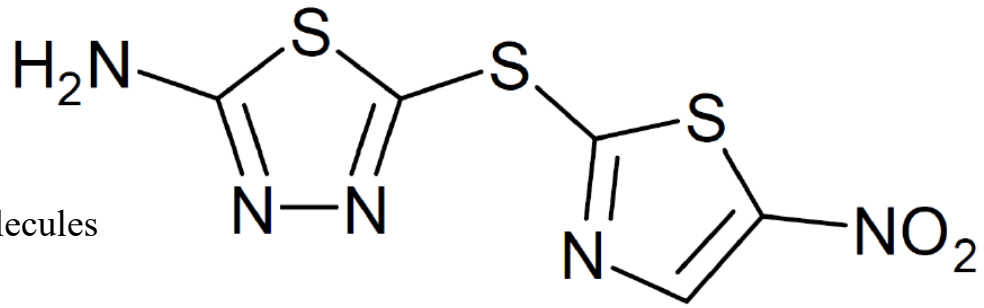


- MLP ignores structure (e.g., spatial and graphical) in the inputs
  - Not ideal for data such as images, text, etc. which are flattened as vectors when used with MLP

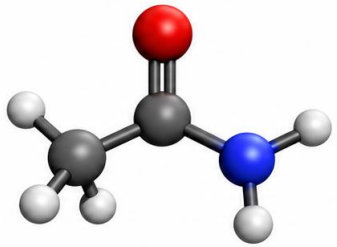
- Fully connected nature of MLP requires massive number of weights
  - Even a "smallish" 200x200x3 (3 channels – R,G,B) image will need 120,000 weights for each neuron in the first hidden layer (for K neurons, we will need 120,000 x K weights).
  - Recall that each layer is fully connected so each layer needs a massive number of weights!

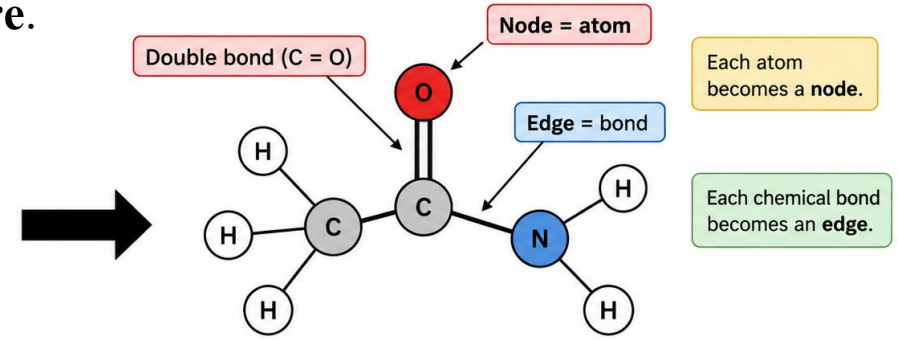
Graphs are *everywhere*!

In many ways, **graphs** are the **main** modality of data we receive from **nature**.

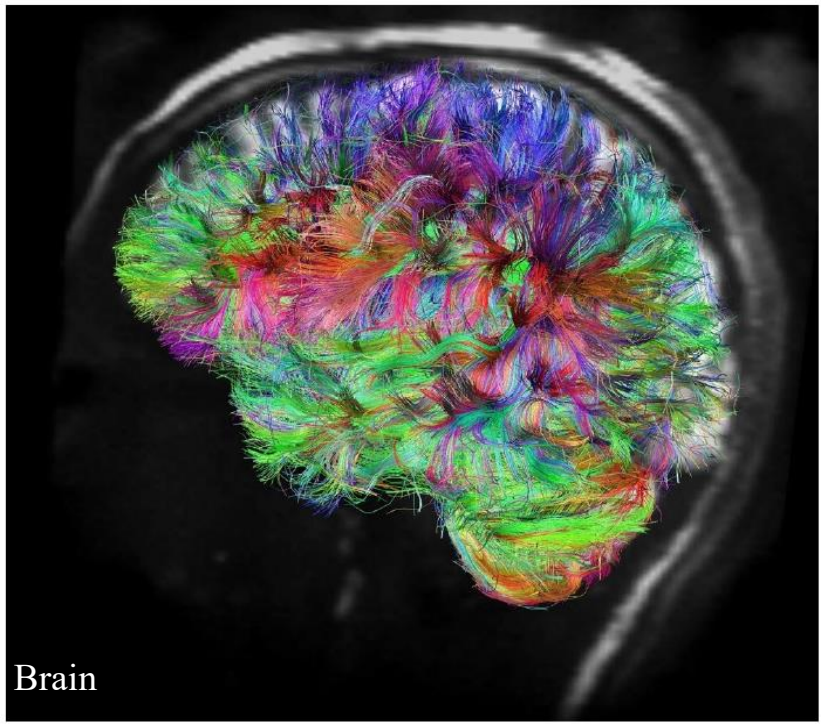


Molecules

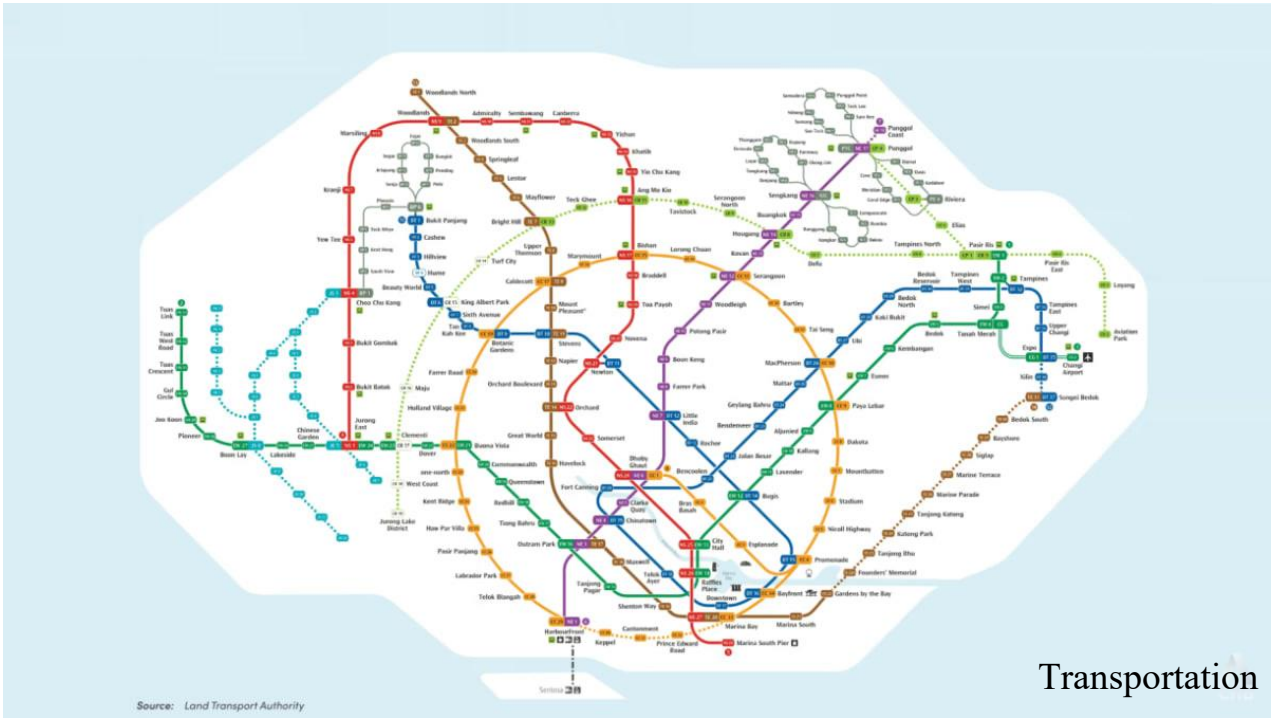




Acetamide molecule as a graph



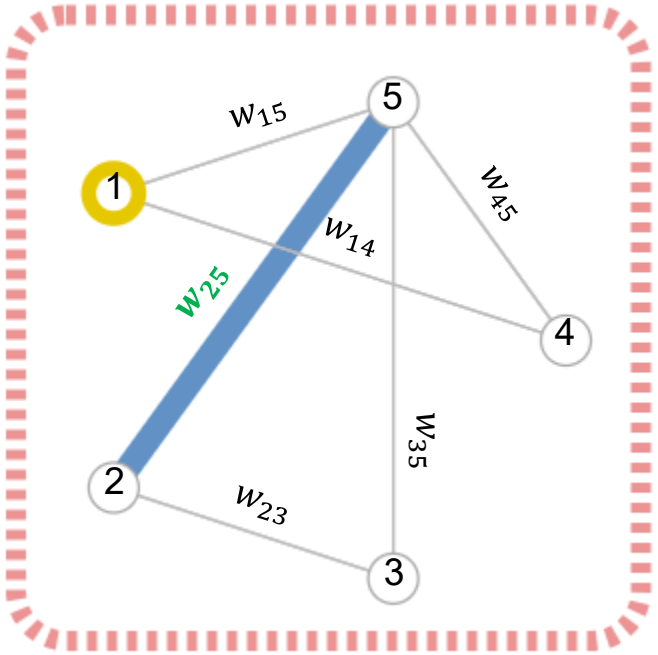
Brain



Transportation

What is a Graph?

- **Graph** represents the relations (edges) between a collection of entities (nodes) quantified using weights.
- **Graph $G(V, E, W)$:** V = nodes, E = edges (link node pairs), W = weights aassociated with edges.
- **Adjacency matrix (A)** of a graph is a sparse matrix with non-zero elements denoting graph weights.



Vertex (or node)

Edges (or link)

Weights

Global context (or master node)

Fig: Attributes of undirected weighted graph

$$A = \begin{pmatrix} 0 & 0 & 0 & w_{14} & w_{15} \\ 0 & 0 & w_{23} & 0 & w_{25} \\ 0 & w_{23} & 0 & 0 & w_{35} \\ w_{14} & 0 & 0 & 0 & w_{45} \\ w_{15} & w_{25} & w_{35} & w_{45} & 0 \end{pmatrix}$$

Adjacency matrix of a symmetric weighted graph

Graph Neural Networks (GNNs)

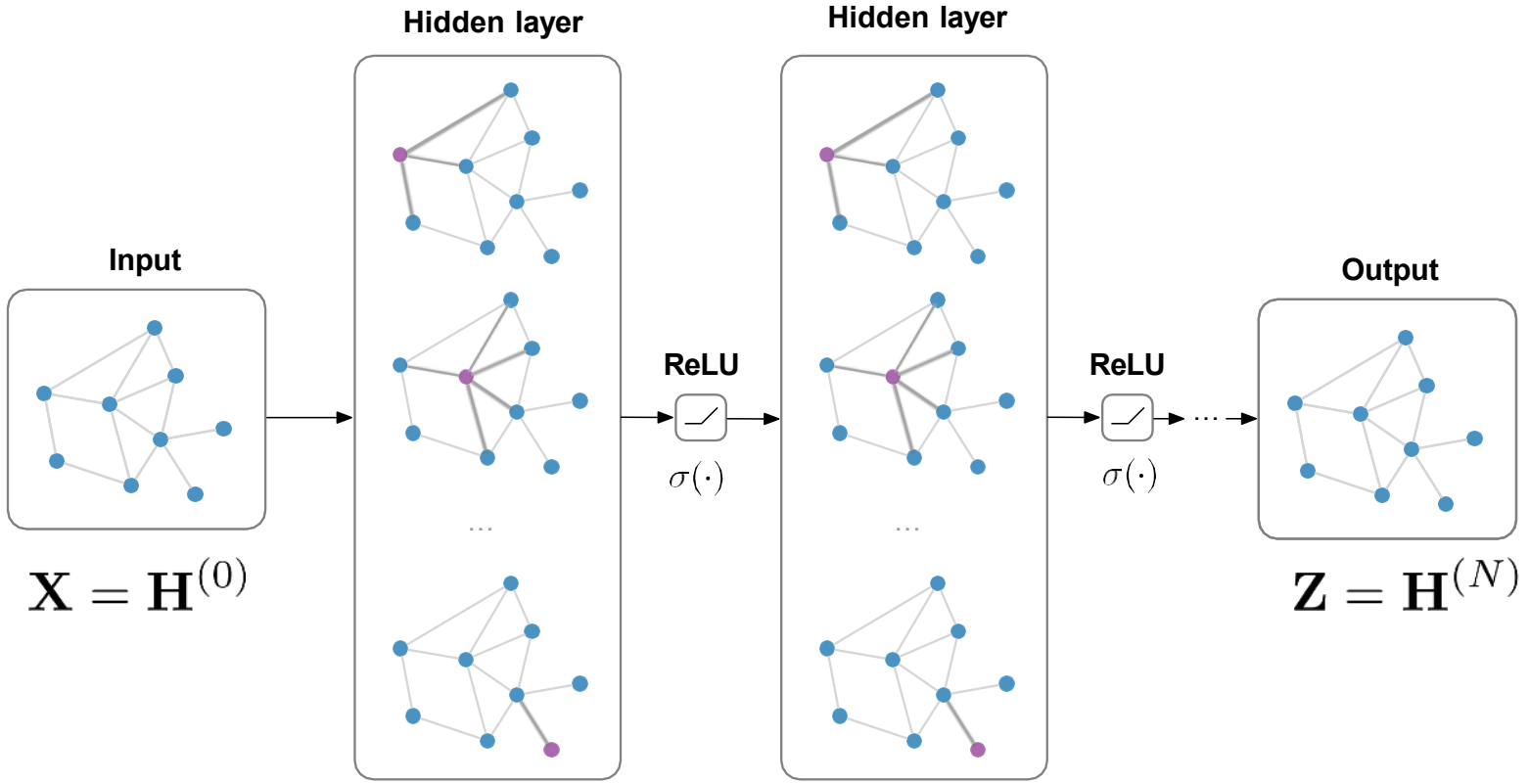
Notation: $\mathcal{G} = (\mathbf{A}, \mathbf{X})$

- Adjacency matrix $\mathbf{A} \in \mathbb{R}^{N \times N}$
- Feature matrix $\mathbf{X} \in \mathbb{R}^{N \times E}$

N = number of **nodes** in the graph
E = number of **features per node**.

Main Idea: Pass massages between pairs of nodes and agglomerate

Alternative Interpretation: Pass massages between nodes to refine node (and possibly edge) representations



Kipf & Welling (ICLR 2017), related previous works by Duvenaud et al. (NIPS 2015) and Li et al. (ICLR 2016)

Neural message passing in GNNs

- GNNs utilize neural message passing network to exchange information amongst neighboring nodes and influence each other's updated embeddings as follows:

i. For each node in the graph, all the neighboring node embeddings (messages) are gathered.

ii. All messages are aggregated via an aggregate function (like sum).

iii. All pooled messages are passed through an update function, usually a learned neural network.

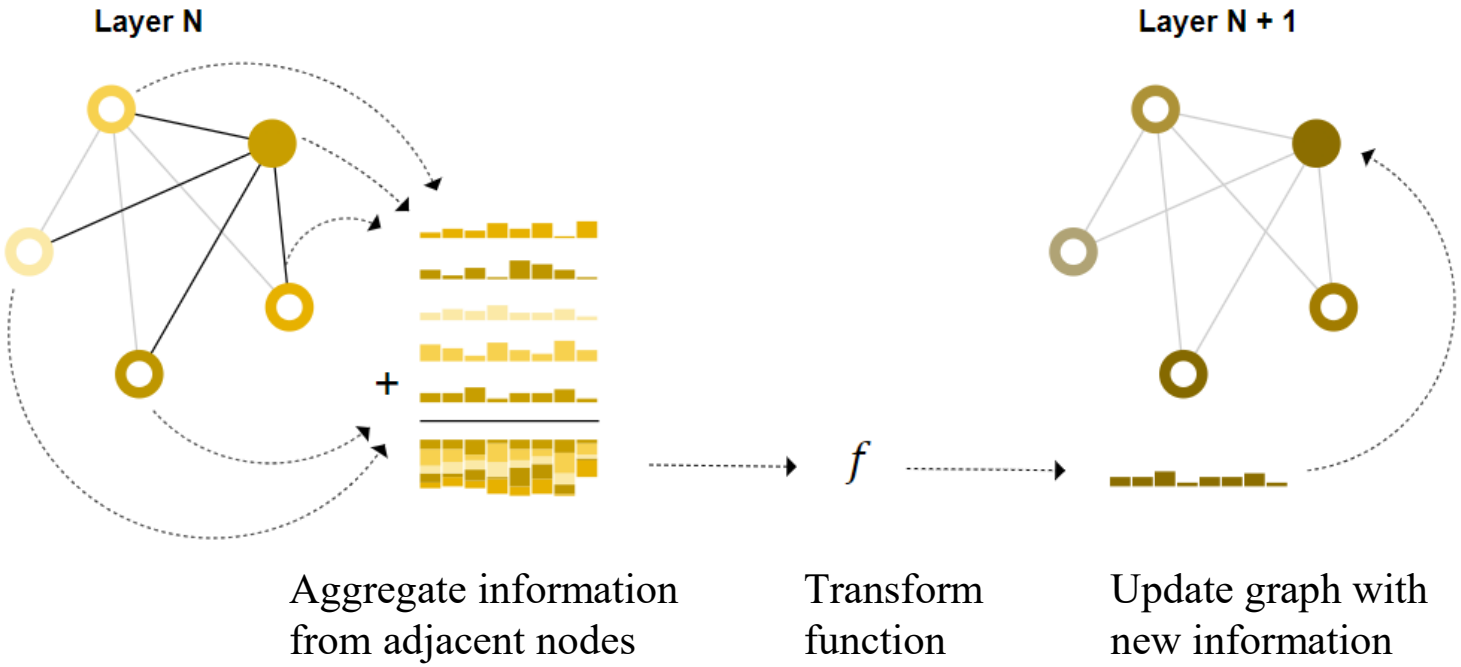
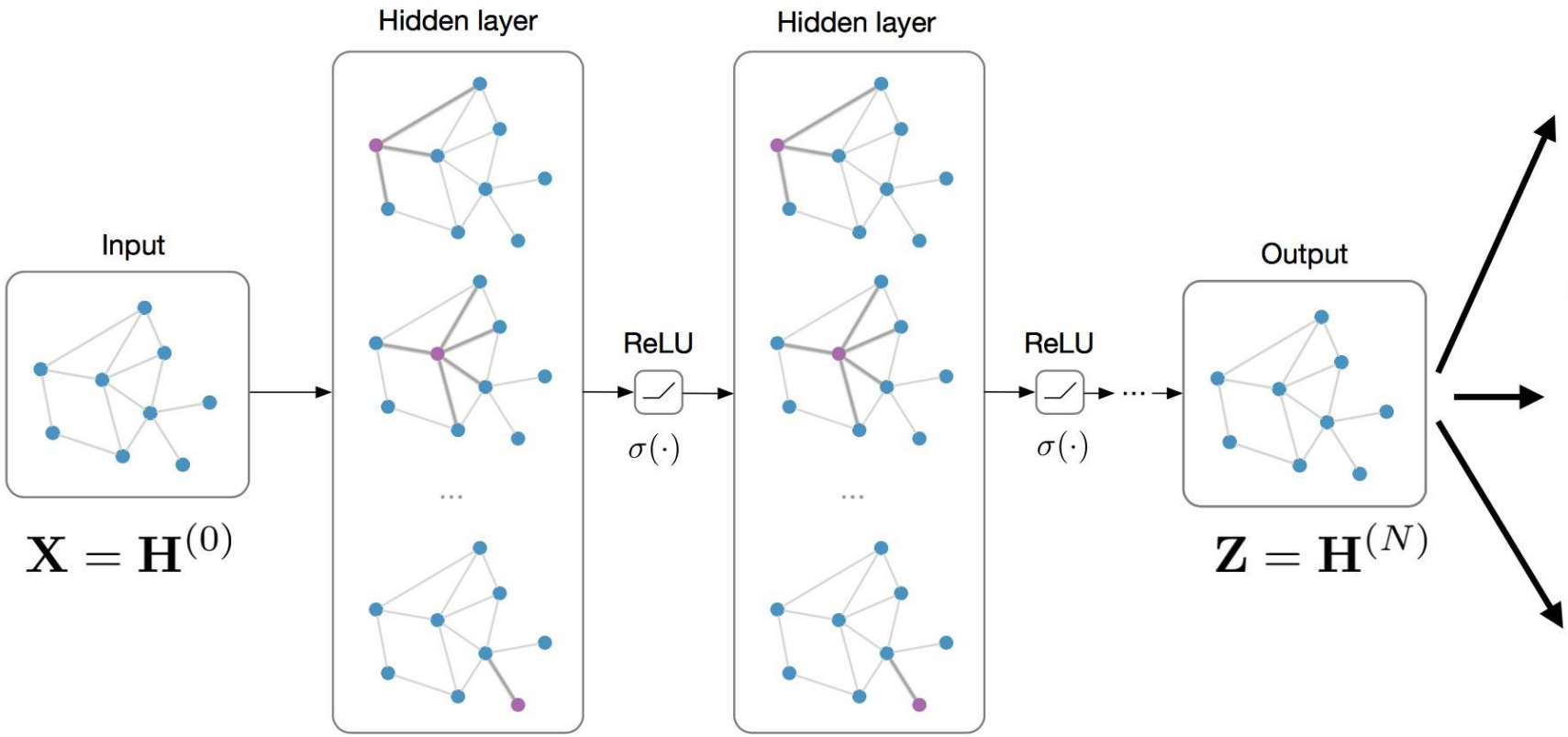


Fig: Updating the highlighted node embedding using adjacent nodes through neural massage passing network.

Classification and Link Prediction with GNNs

Input: Feature matrix $\mathbf{X} \in \mathbb{R}^{N \times E}$, preprocessed adjacency matrix $\hat{\mathbf{A}}$



Node classification:

$\text{softmax}(\mathbf{z_n})$

e.g. Kipf & Welling (ICLR 2017)

Graph classification:

$\text{softmax}(\sum_n \mathbf{z_n})$

e.g. Duvenaud et al. (NIPS 2015)

Link prediction:

$p(A_{ij}) = \sigma(\mathbf{z_i^T z_j})$

Kipf & Welling (NIPS BDL 2016)

"Graph Auto-Encoders"

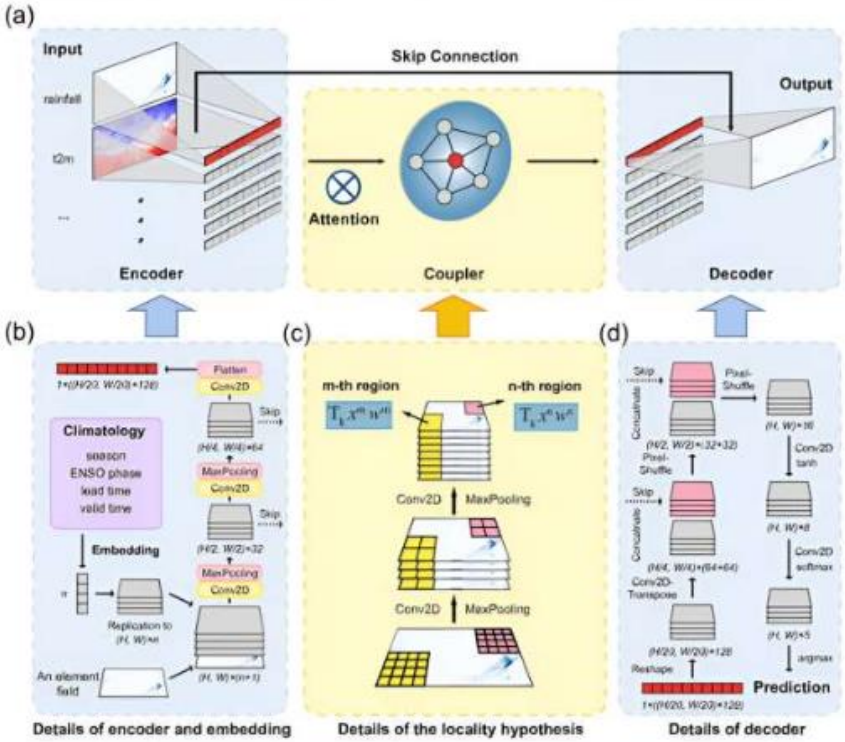
Graph Neural Networks are *everywhere*!

JANUARY 22, 2024

Novel graph neural network models enhance precipitation forecasting

by Chinese Academy of Sciences

Geo. Res. Lett (2024)



VB The Machine GamesBeat Jobs Special Issue Become a Member

The Machine
Making sense of AI

DeepMind claims its AI improved Google Maps travel time estimates by up to 50%

Kyle Wiggers @Kyle_L_Wiggers September 3, 2020 7:00 AM

BBC Sign in News Sport Reel Worklife Travel Future

NEWS

Home Video World UK Business Tech Science Stories Entertainment & Arts

BBC WORKLIFE Our new guide for getting ahead

Scientists discover powerful antibiotic using AI

21 February 2020 Share

Food Discovery with Uber Eats: Using Graph Learning to Power Recommendations

Ankit Jain, Isaac Liu, Ankur Sarda, and Piero Molino December 4, 2019

0



GNNs for Materials Science

communications materials

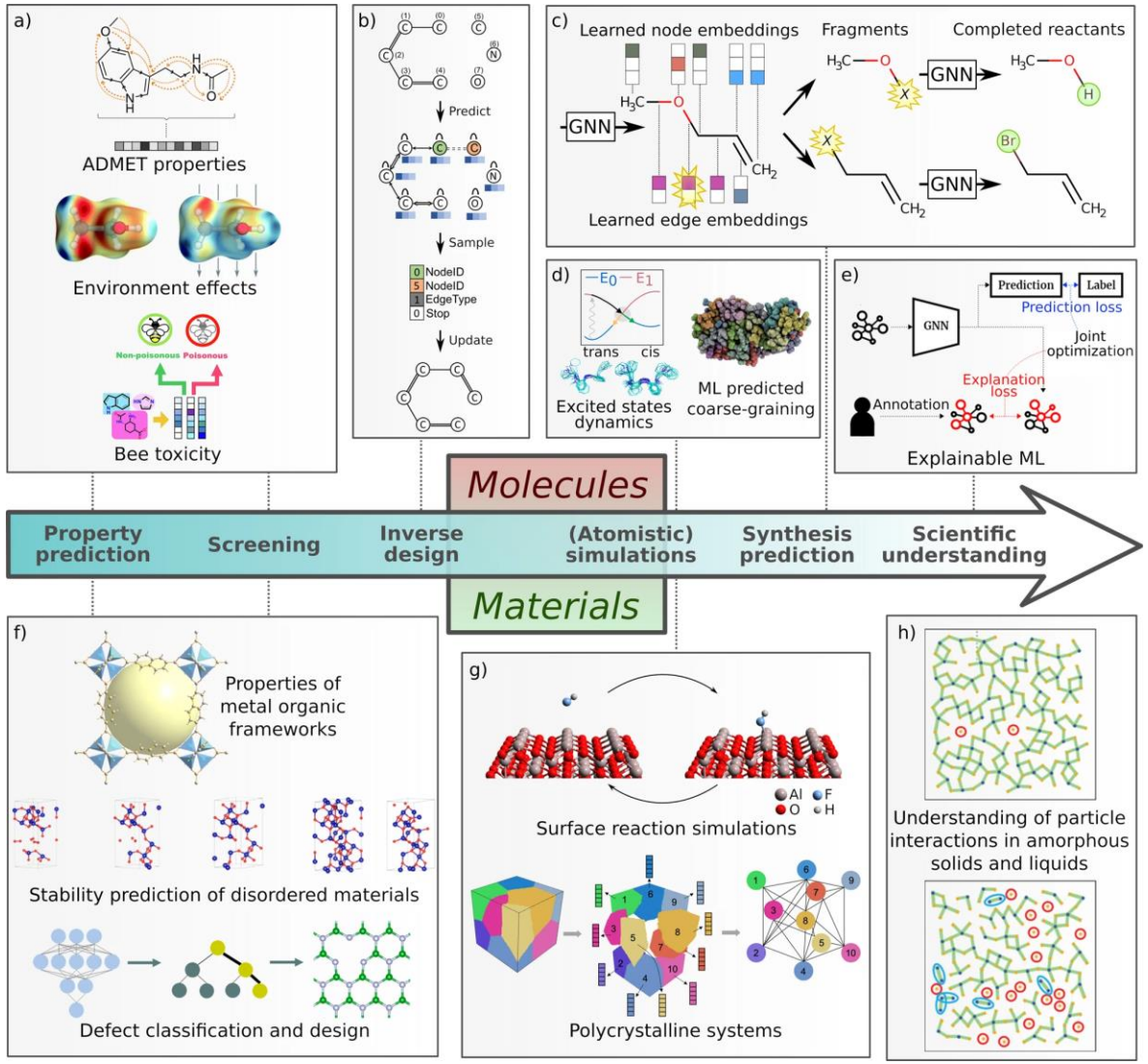
REVIEW ARTICLE

https://doi.org/10.1038/s43246-022-00315-6 OPEN

Graph neural networks for materials science and chemistry

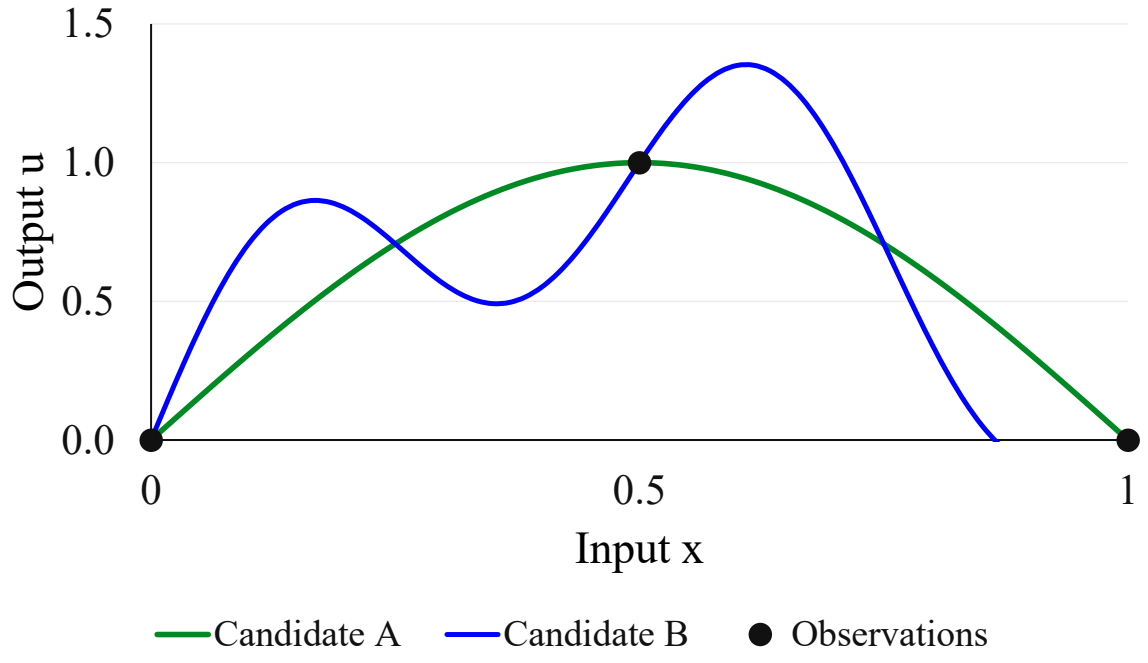
Patrick Reiser[1,2], Marlen Neubert[1], André Eberhard[1], Luca Torresi[1], Chen Zhou[1], Chen Shao[1,6], Houssam Metni[1,3], Clint van Hoesel[1,4], Henrik Schopmans[1,2], Timo Sommer[1,5,7] & Pascal Friederich[1,2]✉

Machine learning plays an increasingly important role in many areas of chemistry and materials science, being used to predict materials properties, accelerate simulations, design new structures, and predict synthesis routes of new materials. Graph neural networks (GNNs) are one of the fastest growing classes of machine learning models. They are of particular relevance for chemistry and materials science, as they directly work on a graph or structural representation of molecules and materials and therefore have full access to all relevant information required to characterize materials. In this Review, we provide an overview of the basic principles of GNNs, widely used datasets, and state-of-the-art architectures, followed by a discussion of a wide range of recent applications of GNNs in chemistry and materials science, and concluding with a road-map for the further development and application of GNNs.



Physics-Informed Neural Networks (PINNs)

- An MLP can fit observations well and still violate a known physical law.
- A PINN trains the network using observations and a governing differential equation.



Several curves fit the same three observations.
Illustration only. These curves are not trained networks.

Known physics supplies training information between the observed points.

GNNs use relationships between entities. PINNs use equations that describe the process.

The MLP architecture can stay the same.
The loss now includes physics.

Example: Heat Diffusion in a Material

Temperature u(x,t) in a homogeneous bar

$$\partial u/\partial t = \alpha\, \partial^2 u/\partial x^2$$

$\alpha > 0$ is the thermal diffusivity.
x is position and t is time.

Initial condition (starting temperature)

$$u(x,0) = \sin(\pi x)$$

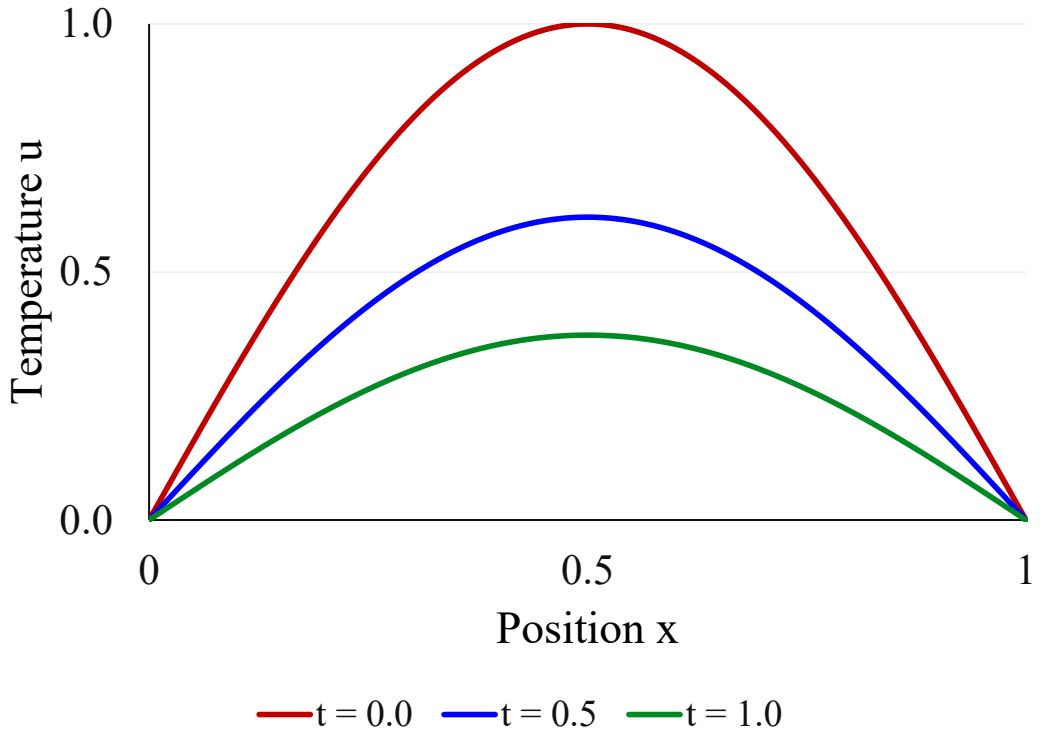
Boundary conditions (cooled ends)

$$u(0,t) = u(1,t) = 0$$

PINN approximation: $\hat{u}(x,t) = \mathrm{MLP}_\theta(x,t)$

Normalized variables: $0 \le x \le 1$ and $0 \le t \le 1$

Reference temperature profiles ($\alpha = 0.1$)

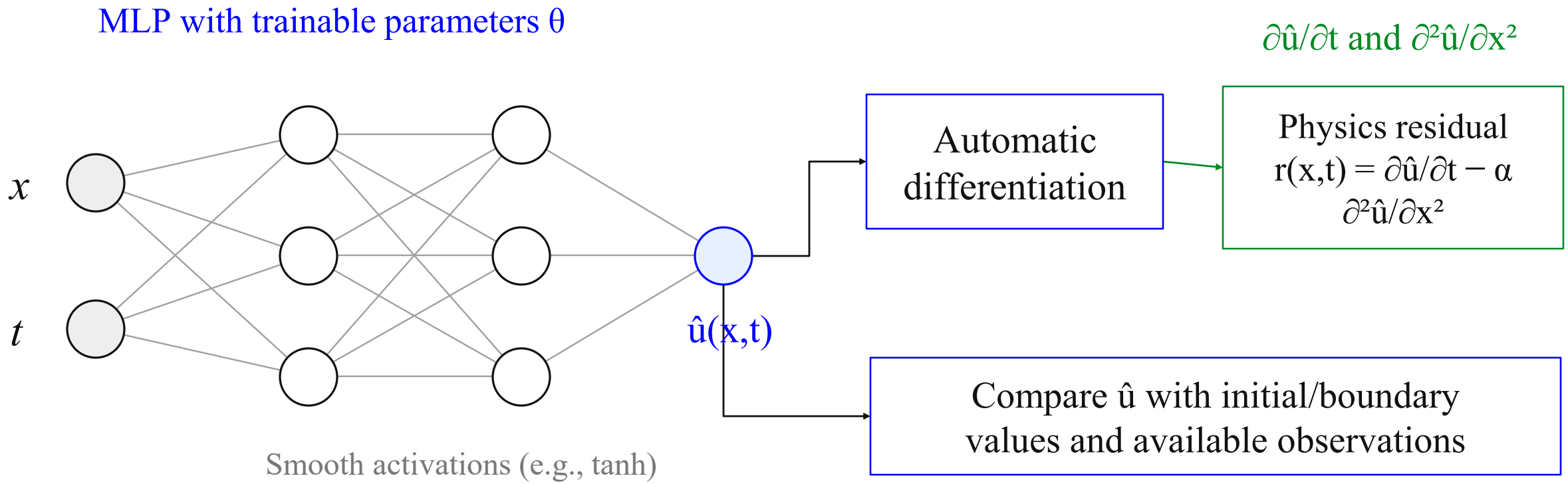


$$u(x,t) = \sin(\pi x)\, \exp(-\alpha \pi^2 t)$$

Exact solution for checking a trained PINN

Automatic Differentiation in a PINN

The MLP maps coordinates (x,t) to a predicted temperature $\hat{u}(x,t)$.



AD computes output derivatives with respect to the input coordinates x and t.

Backpropagation computes the loss gradient with respect to θ to update the network.

The PINN Loss and Training

$$L = \lambda_f L_f + \lambda_0 L_0 + \lambda_b L_b + \lambda_d L_d$$

Physics

$$L_f = (1/N_f) \sum_i r(x_i,t_i)^2$$

Initial condition

$$L_0 = (1/N_0) \sum_i [\hat{u}(x_i,0) - \sin(\pi x_i)]^2$$

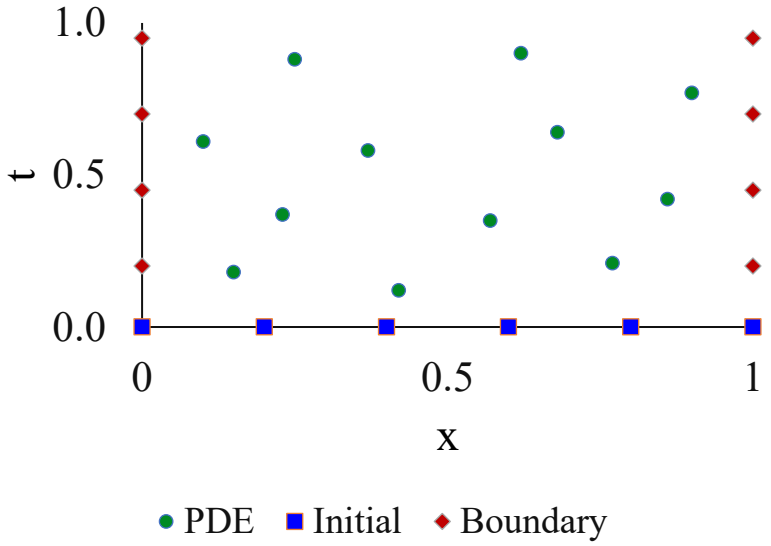
Boundary conditions

$$L_b = (1/N_b) \sum_i [\hat{u}(0,t_i)^2 + \hat{u}(1,t_i)^2]$$

Data (optional)

$$L_d = (1/N_d) \sum_i [\hat{u}(x_i,t_i) - y_i]^2$$

Training points in space and time



Collocation points check the PDE. They do not need measured temperatures.

Choose training points, then minimize L with Adam or L-BFGS.

The λ weights balance the penalties. Set $\lambda_d = 0$ if no interior observations are available.

Forward and Inverse Problems

	Forward problem	**Inverse problem**
Known	Equation, α, initial and boundary conditions	Equation, initial/boundary conditions and observed temperatures
Learn	Network parameters θ to approximate $u(x,t)$	Network parameters θ and the unknown diffusivity $\alpha > 0$
Question	What temperature field follows from these physical conditions?	Which material diffusivity explains the observed temperatures?
Optimize	Minimize L over θ	Minimize L over θ and $\alpha > 0$

Materials science: infer a thermal property from sparse temperature measurements.

Related uses include estimating elastic properties and modeling phase evolution.

Loss penalties encourage physical consistency. Accuracy and parameter identifiability still need checks.

PINNs for Materials Science

Archives of Computational Methods in Engineering (2026) 33:5223–5260
https://doi.org/10.1007/s11831-025-10448-9

REVIEW

Physics-Informed Neural Networks in Materials Modeling and Design: A Review

Alkmini Michaloglou[1] · Ioannis Papadimitriou[1] · Ilias Gialampoukidis[1] · Stefanos Vrochidis[1] · Ioannis Kompatsiaris[1]

Received: 11 July 2025 / Revised: 29 September 2025 / Accepted: 20 October 2025 / Published online: 19 November 2025
© The Author(s) 2025

Abstract

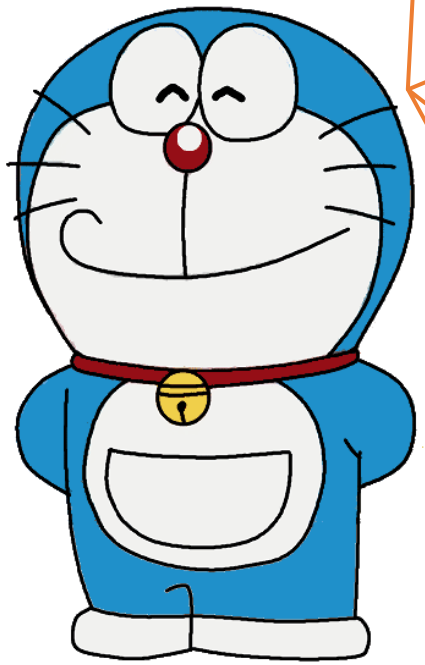
Physics-Informed Neural Networks are neural networks which combine data-driven deep learning and physical models by incorporating physical laws into their loss function. They are promising and flexible solvers or surrogates for problems governed by partial differential equations, including forward and inverse cases, and during the last few years they have been applied to a variety of problems in physics and engineering. In this survey, we present a comprehensive analysis of PINNs while focusing on their applications in materials science. We examine their foundational form and components, expand on enhancing strategies, and explore various aspects of their practical implementations. For materials modeling, problem cases are systematically organized based on the main scope of publications found in the literature. For this survey, problems are categorized into heat transfer related problems, including multiphysics cases, materials properties and design, materials durability, stress analysis, as well as problems related to porous media and multi-phase materials. Finally, we discuss the current challenges and limitations of PINNs, their advantages, and potential future research directions.

Popular Deep Learning Architectures



- MLP: Feedforward fully connected network
  - Not preferred when inputs have spatial/sequential structures (e.g., image, text)
- CNN: Feedforward but NOT fully connected (but last few layers, especially output, are)
- RNNs: Not feedforward (hidden state of one timestep connects with that of the next)
- Transformers: Very powerful models for sequential data
  - Unlike RNNs, can process inputs in parallel. Also uses (self) attention to better capture long range dependencies and context in the input sequence
- Graph Neural Networks: Used when inputs are graphs (e.g., molecules)
- Physics Informed Neural Networks: Used when physical priors are available (materials modelling)
- Autoencoders and Deep Generative Models: For unsupervised earning and synthetic data generation tasks

Any question?



Readings for you:

- Deep Learning book
- Dive into Deep Learning (Very Useful)
- Geometric Deep Learning Book